

(12) **United States Patent**
Bedikian et al.

(10) **Patent No.:** **US 12,405,673 B2**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **FREE-SPACE USER INTERFACE AND CONTROL USING VIRTUAL CONSTRUCTS**

(71) Applicant: **Ultrahaptics IP Two Limited**, Bristol (GB)

(72) Inventors: **Raffi Bedikian**, San Francisco, CA (US); **Jonathan Marsden**, San Mateo, CA (US); **Keith Mertens**, Oakland, CA (US); **David Holz**, San Francisco, CA (US)

(73) Assignee: **Ultrahaptics IP Two Limited**, Bristol (GB)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/506,009**

(22) Filed: **Nov. 9, 2023**

(65) **Prior Publication Data**

US 2024/0077950 A1 Mar. 7, 2024

Related U.S. Application Data

(63) Continuation of application No. 17/833,556, filed on Jun. 6, 2022, now Pat. No. 11,874,970, which is a (Continued)

(51) **Int. Cl.**
G06F 3/01 (2006.01)
G06F 3/03 (2006.01)
(Continued)

(52) **U.S. Cl.**
CPC **G06F 3/017** (2013.01); **G06F 3/011** (2013.01); **G06F 3/0304** (2013.01); **G06F 3/04845** (2013.01); **G06V 40/20** (2022.01)

(58) **Field of Classification Search**
CPC G06F 3/017; G06F 3/011; G06F 3/0304; G06F 3/04845; G06V 40/20
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2,665,041 A 1/1954 Maffucci
3,064,704 A 11/1962 Kim et al.
(Continued)

FOREIGN PATENT DOCUMENTS

CN 1984236 A 6/2007
CN 201332447 Y 10/2009
(Continued)

OTHER PUBLICATIONS

Arthington, et al., "Cross-section Reconstruction During Uniaxial Loading," Measurement Science and Technology, vol. 20, No. 7, Jun. 10, 2009, Retrieved from the Internet: <http://iopscience.iop.org/0957-0233/20/7/075701>, pp. 1-9.

(Continued)

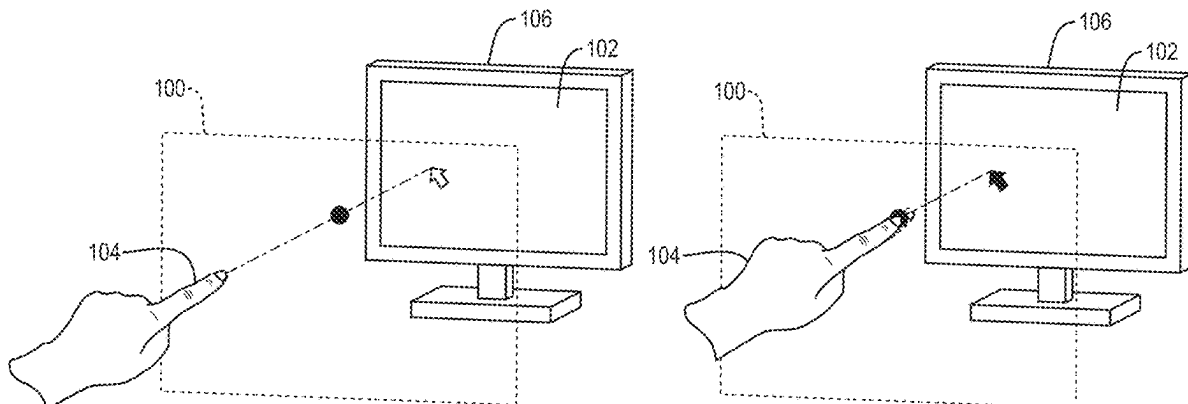
Primary Examiner — Cao H Nguyen

(74) *Attorney, Agent, or Firm* — Haynes Beffel & Wolfeld LLP; Andrew L. Dunlap; Franklin M. Schellenberg

(57) **ABSTRACT**

During control of a user interface via free-space motions of a hand or other suitable control object, switching between control modes can be facilitated by tracking the control object's movements relative to, and its penetration of, a virtual control construct (such as a virtual surface construct). The technology disclosed includes determining from the motion information whether a motion of the control object with respect to the virtual control construct is an engagement gesture, such as a virtual mouse click or other control device operation. The position of the virtual control construct can be updated, continuously or from time to time, based on the control object's location.

19 Claims, 15 Drawing Sheets



Related U.S. Application Data

continuation of application No. 16/987,289, filed on Aug. 6, 2020, now Pat. No. 11,353,962, which is a continuation of application No. 16/054,891, filed on Aug. 3, 2018, now Pat. No. 10,739,862, which is a continuation of application No. 15/358,104, filed on Nov. 21, 2016, now Pat. No. 10,042,430, which is a continuation of application No. 14/154,730, filed on Jan. 14, 2014, now Pat. No. 9,501,152.

- (60) Provisional application No. 61/877,641, filed on Sep. 13, 2013, provisional application No. 61/873,351, filed on Sep. 3, 2013, provisional application No. 61/872,538, filed on Aug. 30, 2013, provisional application No. 61/825,480, filed on May 20, 2013, provisional application No. 61/825,515, filed on May 20, 2013, provisional application No. 61/824,691, filed on May 17, 2013, provisional application No. 61/816,487, filed on Apr. 26, 2013, provisional application No. 61/808,959, filed on Apr. 5, 2013, provisional application No. 61/808,984, filed on Apr. 5, 2013, provisional application No. 61/791,204, filed on Mar. 15, 2013, provisional application No. 61/752,733, filed on Jan. 15, 2013, provisional application No. 61/752,731, filed on Jan. 15, 2013, provisional application No. 61/752,725, filed on Jan. 15, 2013.

(51) **Int. Cl.**

G06F 3/04845 (2022.01)
G06V 40/20 (2022.01)

(56) **References Cited**

U.S. PATENT DOCUMENTS

4,175,862	A	11/1979	DiMatteo et al.
4,876,455	A	10/1989	Sanderson et al.
4,879,659	A	11/1989	Bowlin et al.
4,893,223	A	1/1990	Arnold
5,038,258	A	8/1991	Koch et al.
5,134,661	A	7/1992	Reinsch
5,282,067	A	1/1994	Liu
5,434,617	A	7/1995	Bianchi
5,454,043	A	9/1995	Freeman
5,574,511	A	11/1996	Yang et al.
5,581,276	A	12/1996	Cipolla et al.
5,594,469	A	1/1997	Freeman et al.
5,659,475	A	8/1997	Brown
5,691,737	A	11/1997	Ito et al.
5,742,263	A	4/1998	Wang et al.
5,900,863	A	5/1999	Numazaki
5,940,538	A	8/1999	Spiegel et al.
6,002,808	A	12/1999	Freeman
6,031,161	A	2/2000	Baltenberger
6,031,661	A	2/2000	Tanaami
6,072,494	A	6/2000	Nguyen
6,075,895	A	6/2000	Qiao et al.
6,147,678	A	11/2000	Kumar et al.
6,154,558	A	11/2000	Hsieh
6,181,343	B1	1/2001	Lyons
6,184,326	B1	2/2001	Razavi et al.
6,184,926	B1	2/2001	Khosravi et al.
6,195,104	B1	2/2001	Lyons
6,204,852	B1	3/2001	Kumar et al.
6,252,598	B1	6/2001	Segen
6,263,091	B1	7/2001	Jain et al.
6,346,933	B1	2/2002	Lin
6,417,970	B1	7/2002	Travers et al.
6,463,402	B1	10/2002	Bennett et al.
6,492,986	B1	12/2002	Metaxas et al.
6,493,041	B1	12/2002	Hanko et al.
6,498,628	B2	12/2002	Iwamura
6,578,203	B1	6/2003	Anderson, Jr. et al.
6,603,867	B1	8/2003	Sugino et al.
6,629,065	B1	9/2003	Gadh et al.
6,661,918	B1	12/2003	Gordon et al.
6,674,877	B1	1/2004	Jojic et al.
6,702,494	B2	3/2004	Dumler et al.
6,734,911	B1	5/2004	Lyons
6,738,424	B1	5/2004	Allmen et al.
6,771,294	B1	8/2004	Pulli et al.
6,798,628	B1	9/2004	Macbeth
6,804,654	B2	10/2004	Kobylevsky et al.
6,804,656	B1	10/2004	Rosenfeld et al.
6,814,656	B2	11/2004	Rodriguez
6,819,796	B2	11/2004	Hong et al.
6,901,170	B1	5/2005	Terada et al.
6,919,880	B2	7/2005	Morrison et al.
6,950,534	B2	9/2005	Cohen et al.
6,993,157	B1	1/2006	Oue et al.
7,152,024	B2	12/2006	Marschner et al.
7,213,707	B2	5/2007	Hubbs et al.
7,215,828	B2	5/2007	Luo
7,244,233	B2	7/2007	Krantz et al.
7,257,237	B1	8/2007	Luck et al.
7,259,873	B2	8/2007	Sikora et al.
7,308,112	B2	12/2007	Fujimura et al.
7,340,077	B2	3/2008	Gokturk et al.
7,483,049	B2	1/2009	Aman et al.
7,519,223	B2	4/2009	Dehlin et al.
7,532,206	B2	5/2009	Morrison et al.
7,536,032	B2	5/2009	Bell
7,542,586	B2	6/2009	Johnson
7,598,942	B2	10/2009	Underkoffler et al.
7,606,417	B2	10/2009	Steinberg et al.
7,646,372	B2	1/2010	Marks et al.
7,656,372	B2	2/2010	Sato et al.
7,665,041	B2	2/2010	Wilson et al.
7,692,625	B2	4/2010	Morrison et al.
7,831,932	B2	11/2010	Josephsoon et al.
7,840,031	B2	11/2010	Albertson et al.
7,861,188	B2	12/2010	Josephsoon et al.
7,940,885	B2	5/2011	Stanton et al.
7,948,493	B2	5/2011	Klefenz et al.
7,961,174	B1	6/2011	Markovic et al.
7,961,934	B2	6/2011	Thrun et al.
7,971,156	B2	6/2011	Albertson et al.
7,980,885	B2	7/2011	Gattwinkel et al.
8,023,698	B2	9/2011	Niwa et al.
8,035,624	B2	10/2011	Bell et al.
8,045,825	B2	10/2011	Shimoyama et al.
8,064,704	B2	11/2011	Kim et al.
8,085,339	B2	12/2011	Marks
8,086,971	B2	12/2011	Radivojevic et al.
8,111,239	B2	2/2012	Pryor et al.
8,112,719	B2	2/2012	Hsu et al.
8,144,233	B2	3/2012	Fukuyama
8,185,176	B2	5/2012	Mangat et al.
8,213,707	B2	7/2012	Li et al.
8,218,858	B2	7/2012	Gu
8,229,134	B2	7/2012	Duraiswami et al.
8,235,529	B1	8/2012	Raffle et al.
8,244,233	B2	8/2012	Chang et al.
8,249,345	B2	8/2012	Wu et al.
8,270,669	B2	9/2012	Aichi et al.
8,289,162	B2	10/2012	Mooring et al.
8,290,208	B2	10/2012	Kurtz et al.
8,304,727	B2	11/2012	Lee et al.
8,319,832	B2	11/2012	Nagata et al.
8,363,010	B2	1/2013	Nagata
8,395,600	B2	3/2013	Kawashima et al.
8,432,377	B2	4/2013	Newton
8,471,848	B2	6/2013	Tschesnok
8,514,221	B2	8/2013	King et al.
8,553,037	B2	10/2013	Smith et al.
8,582,809	B2	11/2013	Halimeh et al.
8,593,417	B2	11/2013	Kawashima et al.
8,605,202	B2	12/2013	Muijs et al.
8,631,355	B2	1/2014	Murillo et al.
8,638,989	B2	1/2014	Holz

(56)

References Cited

U.S. PATENT DOCUMENTS

8,659,594	B2	2/2014	Kim et al.	2006/0072105	A1	4/2006	Wagner
8,659,658	B2	2/2014	Vassigh et al.	2006/0098899	A1	5/2006	King et al.
8,693,731	B2	4/2014	Holz et al.	2006/0204040	A1	9/2006	Freeman et al.
8,738,523	B1	5/2014	Sanchez et al.	2006/0210112	A1	9/2006	Cohen et al.
8,744,122	B2	6/2014	Salgian et al.	2006/0262421	A1	11/2006	Matsumoto et al.
8,768,022	B2	7/2014	Miga et al.	2006/0290950	A1	12/2006	Platt et al.
8,803,885	B1	8/2014	Carron	2007/0014466	A1	1/2007	Baldwin
8,817,087	B2	8/2014	Weng et al.	2007/0042346	A1	2/2007	Weller
8,842,084	B2	9/2014	Andersson et al.	2007/0086621	A1	4/2007	Aggarwal et al.
8,843,857	B2	9/2014	Berkes et al.	2007/0130547	A1	6/2007	Boillot
8,854,433	B1	10/2014	Rafii	2007/0206719	A1	9/2007	Suryanarayanan et al.
8,872,914	B2	10/2014	Gobush	2007/0211023	A1	9/2007	Boillot
8,878,749	B1	11/2014	Wu et al.	2007/0230929	A1	10/2007	Niwa et al.
8,891,868	B1	11/2014	Ivanchenko	2007/0238956	A1	10/2007	Haras et al.
8,907,982	B2	12/2014	Zontrop et al.	2008/0013826	A1	1/2008	Hillis et al.
8,922,590	B1	12/2014	Luckett, Jr. et al.	2008/0019576	A1	1/2008	Senftner et al.
8,929,609	B2	1/2015	Padovani et al.	2008/0030429	A1	2/2008	Hailpern et al.
8,930,852	B2	1/2015	Chen et al.	2008/0031492	A1	2/2008	Lanz
8,942,881	B2	1/2015	Hobbs et al.	2008/0056752	A1	3/2008	Denton et al.
8,954,340	B2	2/2015	Sanchez et al.	2008/0064954	A1	3/2008	Adams et al.
8,957,857	B2	2/2015	Lee et al.	2008/0106637	A1	5/2008	Nakao et al.
9,014,414	B2	4/2015	Katano et al.	2008/0106746	A1	5/2008	Shpunt et al.
9,056,396	B1	6/2015	Linnell	2008/0110994	A1	5/2008	Knowles et al.
9,070,019	B2	6/2015	Holz	2008/0111710	A1	5/2008	Boillot
9,119,670	B2	9/2015	Yang et al.	2008/0118091	A1	5/2008	Serfaty et al.
9,122,354	B2	9/2015	Sharma	2008/0126937	A1	5/2008	Pachet
9,124,778	B1	9/2015	Crabtree	2008/0187175	A1	8/2008	Kim et al.
9,182,812	B2	11/2015	Ybanez Zepeda	2008/0244468	A1	10/2008	Nishihara et al.
9,182,838	B2	11/2015	Kikkeri	2008/0246759	A1	10/2008	Summers
9,329,691	B2	5/2016	Izumi	2008/0273764	A1	11/2008	Scholl
9,342,160	B2	5/2016	Bailey et al.	2008/0278589	A1	11/2008	Thorn
9,389,779	B2	7/2016	Anderson et al.	2008/0291160	A1	11/2008	Rabin
9,459,697	B2	10/2016	Bedikian et al.	2008/0304740	A1	12/2008	Sun et al.
9,501,152	B2	11/2016	Bedikian et al.	2008/0319356	A1	12/2008	Cain et al.
9,712,383	B2*	7/2017	Lu H04L 61/5038	2009/0002489	A1	1/2009	Yang et al.
10,042,430	B2	8/2018	Bedikian et al.	2009/0093307	A1	4/2009	Miyaki
10,281,987	B1	5/2019	Yang et al.	2009/0102840	A1	4/2009	Li
10,739,862	B2	8/2020	Bedikian et al.	2009/0103780	A1	4/2009	Nishihara et al.
10,831,281	B2	11/2020	Yang et al.	2009/0116742	A1	5/2009	Nishihara
11,353,962	B2	6/2022	Bedikian et al.	2009/0122146	A1	5/2009	Zalewski et al.
11,567,578	B2	1/2023	Yang et al.	2009/0128564	A1	5/2009	Okuno
11,874,970	B2	1/2024	Bedikian et al.	2009/0153655	A1	6/2009	Ike et al.
2001/0044858	A1	11/2001	Rekimoto	2009/0203993	A1	8/2009	Mangat et al.
2001/0052985	A1	12/2001	Ono	2009/0203994	A1	8/2009	Mangat et al.
2002/0008139	A1	1/2002	Albertelli	2009/0217211	A1*	8/2009	Hildreth G06F 3/048 715/863
2002/0008211	A1	1/2002	Kask	2009/0257623	A1	10/2009	Tang et al.
2002/0021287	A1	2/2002	Tomasi et al.	2009/0274339	A9	11/2009	Cohen et al.
2002/0041327	A1	4/2002	Hildreth et al.	2009/0309710	A1	12/2009	Kakinami
2002/0080094	A1	6/2002	Biocca et al.	2010/0001998	A1	1/2010	Mandella et al.
2002/0105484	A1	8/2002	Navab et al.	2010/0013662	A1	1/2010	Stude
2003/0053658	A1	3/2003	Pavlidis	2010/0013832	A1	1/2010	Xiao et al.
2003/0053659	A1	3/2003	Pavlidis et al.	2010/0020078	A1	1/2010	Shpunt
2003/0081141	A1	5/2003	Mazzapica	2010/0023015	A1	1/2010	Park
2003/0123703	A1	7/2003	Pavlidis et al.	2010/0026963	A1	2/2010	Faulstich
2003/0152289	A1	8/2003	Luo	2010/0027845	A1	2/2010	Kim et al.
2003/0202697	A1	10/2003	Simard et al.	2010/0046842	A1	2/2010	Conwell
2004/0103111	A1	5/2004	Miller et al.	2010/0053164	A1	3/2010	Imai et al.
2004/0125228	A1	7/2004	Dougherty	2010/0053209	A1	3/2010	Rauch et al.
2004/0125984	A1	7/2004	Ito et al.	2010/0053612	A1	3/2010	Ou-Yang et al.
2004/0145809	A1	7/2004	Brenner	2010/0058252	A1	3/2010	Ko
2004/0155877	A1	8/2004	Hong et al.	2010/0066676	A1	3/2010	Kramer et al.
2004/0212725	A1	10/2004	Raskar	2010/0066737	A1	3/2010	Liu
2005/0007673	A1	1/2005	Chaoulov et al.	2010/0066975	A1	3/2010	Rehnstrom
2005/0068518	A1	3/2005	Baney et al.	2010/0091110	A1	4/2010	Hildreth
2005/0094019	A1	5/2005	Grosvenor et al.	2010/0095206	A1	4/2010	Kim
2005/0131607	A1	6/2005	Breed	2010/0118123	A1	5/2010	Freedman et al.
2005/0156888	A1	7/2005	Xie et al.	2010/0121189	A1	5/2010	Ma et al.
2005/0168578	A1	8/2005	Gobush	2010/0125815	A1	5/2010	Wang et al.
2005/0236558	A1	10/2005	Nabeshima et al.	2010/0127995	A1	5/2010	Rigazio et al.
2005/0238201	A1	10/2005	Shamaie	2010/0141762	A1	6/2010	Siann et al.
2006/0017807	A1	1/2006	Lee et al.	2010/0158372	A1	6/2010	Kim et al.
2006/0028656	A1	2/2006	Venkatesh et al.	2010/0162165	A1	6/2010	Addala et al.
2006/0029296	A1	2/2006	King et al.	2010/0177929	A1	7/2010	Kurtz et al.
2006/0034545	A1	2/2006	Mattes et al.	2010/0194863	A1	8/2010	Lopes et al.
2006/0050979	A1	3/2006	Kawahara	2010/0199221	A1	8/2010	Yeung et al.
				2010/0199230	A1	8/2010	Latta et al.
				2010/0199232	A1	8/2010	Mistry et al.
				2010/0201880	A1	8/2010	Iwamura

(56)

References Cited

U.S. PATENT DOCUMENTS

2010/0208942	A1	8/2010	Porter et al.
2010/0219934	A1	9/2010	Matsumoto
2010/0222102	A1	9/2010	Rodriguez
2010/0248836	A1	9/2010	Suzuki et al.
2010/0264833	A1	10/2010	Van Endert et al.
2010/0275159	A1	10/2010	Matsubara et al.
2010/0277411	A1	11/2010	Yee et al.
2010/0296698	A1	11/2010	Lien et al.
2010/0302015	A1	12/2010	Kipman et al.
2010/0302357	A1	12/2010	Hsu et al.
2010/0303298	A1	12/2010	Marks et al.
2010/0306712	A1	12/2010	Snook et al.
2010/0309097	A1	12/2010	Raviv et al.
2010/0321377	A1	12/2010	Gay et al.
2011/0007072	A1	1/2011	Khan et al.
2011/0025818	A1	2/2011	Gallmeier et al.
2011/0026765	A1	2/2011	Ivanich et al.
2011/0043806	A1	2/2011	Guetta et al.
2011/0057875	A1	3/2011	Shigeta et al.
2011/0066984	A1	3/2011	Li
2011/0080337	A1	4/2011	Matsubara et al.
2011/0080470	A1	4/2011	Kuno et al.
2011/0080490	A1	4/2011	Clarkson et al.
2011/0093820	A1	4/2011	Zhang et al.
2011/0107216	A1	5/2011	Bi
2011/0115486	A1	5/2011	Frohlich et al.
2011/0116684	A1	5/2011	Coffman et al.
2011/0119640	A1	5/2011	Berkes et al.
2011/0134112	A1	6/2011	Koh et al.
2011/0148875	A1	6/2011	Kim et al.
2011/0169726	A1	7/2011	Holmdahl et al.
2011/0173574	A1	7/2011	Clavin et al.
2011/0176146	A1	7/2011	Alvarez Diez et al.
2011/0181509	A1	7/2011	Rautiainen et al.
2011/0193778	A1	8/2011	Lee et al.
2011/0205151	A1	8/2011	Newton et al.
2011/0213664	A1	9/2011	Osterhout et al.
2011/0228978	A1	9/2011	Chen et al.
2011/0234840	A1	9/2011	Klefenz et al.
2011/0243451	A1	10/2011	Oyaizu
2011/0251896	A1	10/2011	Impollonia et al.
2011/0261178	A1	10/2011	Lo et al.
2011/0267259	A1	11/2011	Tidemand et al.
2011/0267265	A1	11/2011	Stinson
2011/0279397	A1	11/2011	Rimon et al.
2011/0286676	A1	11/2011	El Dokor
2011/0289455	A1	11/2011	Reville et al.
2011/0289456	A1	11/2011	Reville et al.
2011/0291925	A1	12/2011	Israel et al.
2011/0291988	A1	12/2011	Bamji et al.
2011/0296353	A1	12/2011	Ahmed et al.
2011/0299737	A1	12/2011	Wang et al.
2011/0304600	A1	12/2011	Yoshida
2011/0304650	A1	12/2011	Campillo et al.
2011/0310007	A1	12/2011	Margolis et al.
2011/0310220	A1	12/2011	McEldowney
2011/0314427	A1	12/2011	Sundararajan
2011/0317871	A1	12/2011	Tossell et al.
2012/0038637	A1	2/2012	Marks
2012/0050157	A1	3/2012	Latta et al.
2012/0065499	A1	3/2012	Chono
2012/0068914	A1	3/2012	Jacobsen et al.
2012/0098744	A1	4/2012	Stinson, III
2012/0113223	A1	5/2012	Hilliges et al.
2012/0113316	A1	5/2012	Ueta et al.
2012/0159380	A1	6/2012	Kocienda et al.
2012/0163675	A1	6/2012	Joo et al.
2012/0194517	A1	8/2012	Izadi et al.
2012/0204133	A1*	8/2012	Guendelman
2012/0218263	A1	8/2012	Meier et al.
2012/0223959	A1	9/2012	Lengeling
2012/0236288	A1	9/2012	Stanley
2012/0250936	A1	10/2012	Holmgren
2012/0270654	A1	10/2012	Padovani et al.
2012/0274781	A1	11/2012	Shet et al.
2012/0281873	A1	11/2012	Brown et al.
2012/0293667	A1	11/2012	Baba et al.
2012/0314030	A1	12/2012	Datta et al.
2012/0320080	A1	12/2012	Giese et al.
2013/0019204	A1	1/2013	Kotler et al.
2013/0033483	A1	2/2013	Im et al.
2013/0038694	A1	2/2013	Nichani et al.
2013/0044951	A1	2/2013	Cherng et al.
2013/0050425	A1	2/2013	Im et al.
2013/0057469	A1	3/2013	Ajika
2013/0086531	A1	4/2013	Sugita et al.
2013/0097566	A1	4/2013	Berglund
2013/0120319	A1	5/2013	Givon
2013/0148852	A1	6/2013	Partis et al.
2013/0181897	A1	7/2013	Izumi
2013/0182079	A1	7/2013	Holz
2013/0182897	A1	7/2013	Holz
2013/0187952	A1	7/2013	Berkovich et al.
2013/0191911	A1	7/2013	Dellinger et al.
2013/0194173	A1	8/2013	Zhu et al.
2013/0208948	A1	8/2013	Berkovich et al.
2013/0222233	A1	8/2013	Park et al.
2013/0222640	A1	8/2013	Baek et al.
2013/0239059	A1	9/2013	Chen et al.
2013/0241832	A1	9/2013	Rimon et al.
2013/0252691	A1	9/2013	Alexopoulos
2013/0257736	A1	10/2013	Hou et al.
2013/0258140	A1	10/2013	Lipson et al.
2013/0271397	A1	10/2013	MacDougall et al.
2013/0283213	A1	10/2013	Guendelman et al.
2013/0300831	A1	11/2013	Mavromatis et al.
2013/0307935	A1	11/2013	Rappel et al.
2013/0321265	A1	12/2013	Bychkov et al.
2014/0002365	A1	1/2014	Ackley et al.
2014/0010441	A1	1/2014	Shamaie
2014/0015831	A1	1/2014	Kim et al.
2014/0037135	A1	2/2014	Kutliroff et al.
2014/0055385	A1	2/2014	Duheille
2014/0055396	A1	2/2014	Aubauer et al.
2014/0063055	A1	3/2014	Osterhout et al.
2014/0063060	A1	3/2014	Maciocci et al.
2014/0064566	A1	3/2014	Shreve et al.
2014/0071069	A1	3/2014	Anderson et al.
2014/0081521	A1	3/2014	Frojd et al.
2014/0085203	A1	3/2014	Kobayashi
2014/0095119	A1	4/2014	Lee et al.
2014/0098018	A1	4/2014	Kim et al.
2014/0125775	A1	5/2014	Holz
2014/0125813	A1	5/2014	Holz
2014/0132738	A1	5/2014	Ogura et al.
2014/0134733	A1	5/2014	Wu et al.
2014/0139425	A1	5/2014	Sakai
2014/0139641	A1	5/2014	Holz
2014/0157135	A1	6/2014	Lee et al.
2014/0161311	A1	6/2014	Kim
2014/0168062	A1	6/2014	Katz et al.
2014/0176420	A1	6/2014	Zhou et al.
2014/0177913	A1	6/2014	Holz
2014/0189579	A1	7/2014	Rimon et al.
2014/0192024	A1	7/2014	Holz
2014/0201666	A1	7/2014	Bedikian et al.
2014/0201689	A1	7/2014	Bedikian et al.
2014/0222385	A1	8/2014	Muenster et al.
2014/0223385	A1	8/2014	Ton et al.
2014/0225826	A1	8/2014	Juni
2014/0225918	A1	8/2014	Mittal et al.
2014/0236529	A1	8/2014	Gyorfi
2014/0240215	A1	8/2014	Tremblay et al.
2014/0240225	A1	8/2014	Eilat
2014/0248950	A1	9/2014	Tosas Bautista
2014/0249961	A1	9/2014	Zagel et al.
2014/0253512	A1	9/2014	Narikawa et al.
2014/0253785	A1	9/2014	Chan et al.
2014/0267098	A1	9/2014	Na et al.
2014/0282282	A1	9/2014	Holz
2014/0307920	A1	10/2014	Holz
2014/0320408	A1	10/2014	Zagorsek et al.
2014/0344762	A1	11/2014	Grasset et al.

G06F 3/017
715/863

(56) References Cited

U.S. PATENT DOCUMENTS

2014/0364209	A1	12/2014	Perry
2014/0364212	A1	12/2014	Osman et al.
2014/0369558	A1	12/2014	Holz
2014/0375547	A1	12/2014	Katz et al.
2015/0003673	A1	1/2015	Fletcher
2015/0009149	A1	1/2015	Gharib et al.
2015/0016777	A1	1/2015	Abovitz et al.
2015/0022447	A1	1/2015	Hare et al.
2015/0029091	A1	1/2015	Nakashima et al.
2015/0040040	A1	2/2015	Balan et al.
2015/0054729	A1	2/2015	Minnen et al.
2015/0084864	A1	3/2015	Geiss et al.
2015/0097772	A1	4/2015	Starner
2015/0103004	A1	4/2015	Cohen et al.
2015/0115802	A1	4/2015	Kuti et al.
2015/0116214	A1	4/2015	Grunnet-Jepsen et al.
2015/0131859	A1	5/2015	Kim et al.
2015/0153833	A1	6/2015	Pinault et al.
2015/0172539	A1	6/2015	Neglur
2015/0193669	A1	7/2015	Gu et al.
2015/0205358	A1	7/2015	Lyren
2015/0205400	A1	7/2015	Hwang et al.
2015/0206321	A1	7/2015	Scavezze et al.
2015/0227795	A1	8/2015	Starner et al.
2015/0234569	A1	8/2015	Hess
2015/0253428	A1	9/2015	Holz
2015/0258432	A1	9/2015	Stafford et al.
2015/0261291	A1	9/2015	Mikhailov et al.
2015/0293597	A1	10/2015	Mishra et al.
2015/0304593	A1	10/2015	Sakai
2015/0309629	A1	10/2015	Amariutei et al.
2015/0323785	A1	11/2015	Fukata et al.
2015/0363070	A1	12/2015	Katz
2016/0062573	A1	3/2016	Dascola et al.
2016/0086046	A1	3/2016	Holz et al.
2016/0093105	A1	3/2016	Rimon et al.
2017/0102791	A1	4/2017	Hosenpud et al.
2019/0073112	A1	3/2019	Holz

FOREIGN PATENT DOCUMENTS

CN	101729808	A	6/2010
CN	101930610	A	12/2010
CN	101951474	A	1/2011
CN	102053702	A	5/2011
CN	201859393	U	6/2011
CN	102201121	A	9/2011
CN	102236412	A	11/2011
DE	4201934	A1	7/1993
DE	10326035	A1	1/2005
DE	102007015495	A1	10/2007
DE	102007015497	B4	1/2014
EP	0999542	A1	5/2000
EP	1477924	A2	11/2004
EP	1837665	A2	9/2007
EP	2369443	A2	9/2011
EP	2378488	A2	10/2011
GB	2419433	A	4/2006
GB	2480140	A	11/2011
GB	2519418	A	4/2015
JP	H02236407	A	9/1990
JP	H08261721	A	10/1996
JP	H09259278	A	10/1997
JP	2000023038	A	1/2000
JP	2002133400	A	5/2002
JP	2003256814	A	9/2003
JP	2004246252	A	9/2004
JP	2006019526	A	1/2006
JP	2006259829	A	9/2006
JP	2007272596	A	10/2007
JP	2008227569	A	9/2008
JP	2009031939	A	2/2009
JP	2009037594	A	2/2009
JP	2010060548	A	3/2010
JP	2011010258	A	1/2011

JP	2011065652	A	3/2011
JP	2011107681	A	6/2011
JP	4906960	B2	3/2012
JP	2012527145	A	11/2012
KR	101092909	B1	12/2011
RU	2422878	C1	6/2011
TW	200844871	A	11/2008
WO	9426057	A1	11/1994
WO	2004114220	A1	12/2004
WO	2006020846	A2	2/2006
WO	2007137093	A2	11/2007
WO	2010007662	A1	1/2010
WO	2010032268	A2	3/2010
WO	2010076622	A1	7/2010
WO	2010088035	A2	8/2010
WO	2010138741	A1	12/2010
WO	2010148155	A2	12/2010
WO	2011024193	A2	3/2011
WO	2011036618	A2	3/2011
WO	2011044680	A1	4/2011
WO	2011045789	A1	4/2011
WO	2011119154	A1	9/2011
WO	2012027422	A2	3/2012
WO	2012039140	A1	3/2012
WO	2013109608	A2	7/2013
WO	2013109609	A2	7/2013
WO	2014208087	A1	12/2014
WO	2015026707	A1	2/2015

OTHER PUBLICATIONS

Ballan et al., "Lecture Notes Computer Science: 12th European Conference on Computer Vision: Motion Capture of Hands in Action Using Discriminative Salient Points", Oct. 7-13, 2012 [retrieved Jul. 14, 2016], Springer Berlin Heidelberg, vol. 7577, pp. 640-653. Retrieved from the Internet: <http://link.springer.com/chapter/10.1007/978-3-642-33783-3_46>.

Barat et al., "Feature Correspondences From Multiple Views of Coplanar Ellipses", 2nd International Symposium on Visual Computing, Author Manuscript, 2006, 10 pages.

Bardinet, et al., "Fitting of iso-Surfaces Using Superquadrics and Free-Form Deformations" [on-line], Jun. 24-25, 1994 [retrieved Jan. 9, 2014], 1994 Proceedings of IEEE Workshop on Biomedical Image Analysis, Retrieved from the Internet: http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=315882&tag=1, pp. 184-193.

Butail, S., et al., "Three-Dimensional Reconstruction of the Fast-Start Swimming Kinematics of Densely Schooling Fish," Journal of the Royal Society Interface, Jun. 3, 2011, retrieved from the Internet <<http://www.ncbi.nlm.nih.gov/pubmed/21642367>>, pp. 0, 1-12.

Cheikh et al., "Multipeople Tracking Across Multiple Cameras", International Journal on New Computer Architectures and Their Applications (IJNCAA), vol. 2, No. 1, 2012, pp. 23-33.

Chung, et al., "Recovering LSHGCs and SHGCs from Stereo," International Journal of Computer Vision, vol. 20, No. 1/2, 1996, pp. 43-58.

Cui et al., "Applications of Evolutionary Computing: Vision-Based Hand Motion Capture Using Genetic Algorithm", 2004 [retrieved Jul. 15, 2016], Springer Berlin Heidelberg, vol. 3005 of LNCS, pp. 289-300. Retrieved from the Internet: <http://link.springer.com/chapter/10.1007/978-3-540-24653-4_30>.

Cumani, A., et al., "Recovering the 3D Structure of Tubular Objects from Stereo Silhouettes," Pattern Recognition, Elsevier, GB, vol. 30, No. 7, Jul. 1, 1997, 9 pages.

Davis et al., "Toward 3-D Gesture Recognition", International Journal of Pattern Recognition and Artificial Intelligence, vol. 13, No. 03, 1999, pp. 381-393.

Delamarre et al., "Finding Pose of Hand in Video Images: A Stereo-based Approach", Apr. 14-16, 1998 [retrieved Jul. 15, 2016], Third IEEE Intern Conf on Auto Face and Gesture Recog, pp. 585-590. Retrieved from the Internet: <http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=671011&url=http%3A%2F%2Fieeexplore.ieee.org%2Fxppls%2Fabs_all.jsp%3Farnumber%3D671011>.

(56)

References Cited**OTHER PUBLICATIONS**

Di Zeno, S., et al., "Advances in Image Segmentation," Image and Vision Computing, Elsevier, Guildford, GBN, vol. 1, No. 1, Copyright Butterworth & Co Ltd., Nov. 1, 1983, pp. 196-210.

Dombeck, D., et al., "Optical Recording of Action Potentials with Second-Harmonic Generation Microscopy," The Journal of Neuroscience, Jan. 28, 2004, vol. 24(4): pp. 999-1003.

Forbes, K., et al., "Using Silhouette Consistency Constraints to Build 3D Models," University of Cape Town, Copyright De Beers 2003, Retrieved from the internet: <<http://www.dip.ee.uct.ac.za/~kforbes/Publications/Forbes2003Prasa.pdf>> on Jun. 17, 2013, 6 pages.

Fukui et al. "Multiple Object Tracking System with Three Level Continuous Processes" IEEE, 1992, pp. 19-27.

Gorce et al., "Model-Based 3D Hand Pose Estimation from Monocular Video", Feb. 24, 2011 [retrieved Jul. 15, 2016], IEEE Transac Pattern Analysis and Machine Intell, vol. 33, Issue: 9, pp. 1793-1805, Retri Internet: <<http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=5719617&url=http%3A%2F%2Fieeexplore.ieee.org%2Fxppls%2Fabsall.jsp%3Farnumber%3D5719617>>.

Guo et al., Featured Wand for 3D Interaction, Jul. 2-5, 2007 [retrieved Jul. 15, 2016], 2007 IEEE International Conference on Multimedia and Expo, pp. 2230-2233. Retrieved from the Internet: <<http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=4285129&tag=1&url=http%3A%2F%2Fieeexplore.ieee.org%2Fxppls%2Fabsall.jsp%3Farnumber%3D4285129%26tag%3D1>>.

Heikkila, J., "Accurate Camera Calibration and Feature Based 3-D Reconstruction from Monocular Image Sequences", Infotech Oulu and Department of Electrical Engineering, University of Oulu, 1997, 126 pages.

Kanhangad, V., et al., "A Unified Framework for Contactless Hand Verification," IEEE Transactions on Information Forensics and Security, IEEE, Piscataway, NJ, US, vol. 6, No. 3, Sep. 1, 2011, pp. 1014-1027.

Kim, et al., "Development of an Orthogonal Double-Image Processing Algorithm to Measure Bubble," Department of Nuclear Engineering and Technology, Seoul National University Korea, vol. 39 No. 4, Published Jul. 6, 2007, pp. 313-326.

Kulesza, et al., "Arrangement of a Multi Stereo Visual Sensor System for a Human Activities Space," Source: Stereo Vision, Book edited by: Dr. Asim Bhatti, ISBN 978-953-7619-22-0, Copyright Nov. 2008, I-Tech, Vienna, Austria, www.intechopen.com, pp. 153-173.

Matsuyama et al. "Real-Time Dynamic 3-D Object Shape Reconstruction and High-Fidelity Texture Mapping for 3-D Video," IEEE Transactions on Circuits and Systems for Video Technology, vol. 14, No. 3, Mar. 2004, pp. 357-369.

May, S., et al., "Robust 3D-Mapping with Time-of-Flight Cameras," 2009 IEEE/RSJ International Conference on Intelligent Robots and Systems, Piscataway, NJ, USA, Oct. 10, 2009, pp. 1673-1678.

Melax et al., "Dynamics Based 3D Skeletal Hand Tracking", May 29, 2013 [retrieved Jul. 14, 2016], Proceedings of Graphics Interface, 2013, pp. 63-70. Retrieved from the Internet: <<http://dl.acm.org/citation.cfm?id=2532141>>.

Mendez, et al., "Importance Masks for Revealing Occluded Objects in Augmented Reality," Proceedings of the 16th ACM Symposium on Virtual Reality Software and Technology, 2 pages, ACM, 2009.

Oka et al., "Real-Time Fingertip Tracking and Gesture Recognition", Nov./Dec. 2002 [retrieved Jul. 15, 2016], IEEE Computer

Graphics and Applications, vol. 22, Issue: 6, pp. 64-71. Retrieved from the Internet: <<http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=1046630&url=http%3A%2F%2Fieeexplore.ieee.org%2Fxppls%2Fabsall.jsp%3Farnumber%3D1046630>>.

Olsson, K., et al., "Shape from Silhouette Scanner—Creating a Digital 3D Model of a Real Object by Analyzing Photos From Multiple Views," University of Linköping, Sweden, Copyright VCG 2001, Retrieved from the Internet: <<http://liu.diva-portal.org/smash/get/diva2:18671/FULLTEXT01>> on Jun. 17, 2013, 52 pages.

Pavlovic, V.I., et al., "Visual Interpretation of Hand Gestures for Human-Computer Interaction: A Review," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 19, No. 7, Jul. 1997, pp. 677-695.

Pedersini, et al., Accurate Surface Reconstruction from Apparent Contours, Sep. 5-8, 2000 European Signal Processing Conference EUSIPCO 2000, vol. 4, Retrieved from the Internet: http://home.deib.polimi.it/sarti/CV_and_publications.html, pp. 1-4.

Rasmussen, Matthew K., "An Analytical Framework for the Preparation and Animation of a Virtual Mannequin for the Purpose of Mannequin-Clothing Interaction Modeling", A Thesis Submitted in Partial Fulfillment of the Requirements for the Master of Science Degree in Civil and Environmental Engineering in the Graduate College of the University of Iowa, Dec. 2008, 98 pages.

Schaar, R., VCNL4020 Vishay Semiconductors. Application Note [online]. Extended Detection Range with VCNL Family of Proximity Sensor Vishay Intertechnology, Inc, Revised Oct. 25, 2013 [retrieved Mar. 4, 2014]. Retrieved from the Internet: <www.vishay.com>. 4 pages.

Schlattmann et al., "Markerless 4 gestures 6 DOF real-time visual tracking of the human hand with automatic initialization", 2007 [retrieved Jul. 15, 2016], Eurographics 2007, vol. 26, No. 3, 10 pages, Retrieved from the Internet: <<http://cg.cs.uni-bonn.de/aigaion2root/attachments/schlattmann-2007-markerless.pdf>>.

Texas Instruments, "4-Channel, 12-Bit, 80-MSPS ADC," VSP5324, Revised Nov. 2012, Texas Instruments Incorporated, 55 pages.

Texas Instruments, "QVGA 3D Time-of-Flight Sensor," Product Overview: OPT 8140, Dec. 2013, Texas Instruments Incorporated, 10 pages.

Texas Instruments, "Time-of-Flight Controller (TFC)," Product Overview: OPT9220, Jan. 2014, Texas Instruments Incorporated, 43 pages.

VCNL4020 Vishay Semiconductors. Datasheet [online]. Vishay Intertechnology, Inc, Doc No. 83476, Rev. 1.3, Oct. 29, 2013 [retrieved Mar. 4, 2014]. Retrieved from the Internet: <www.vishay.com>. 16 pages.

Wang et al., "Tracking of Deformable Hand in Real Time as Continuous Input for Gesture-based Interaction", Jan. 28, 2007 [retrieved Jul. 15, 2016], Proceedings of the 12th International Conference on Intelligent User Interfaces, pp. 235-242. Retrieved from the Internet: <<http://dl.acm.org/citation.cfm?id=1216338>>.

Wu, Y., et al., "Vision-Based Gesture Recognition: A Review," Beckman Institute, Copyright 1999, pp. 103-115.

Zhao et al., "Combining Marker-Based Mocap and RGB-D Camera for Acquiring High-Fidelity Hand Motion Data", Jul. 29, 2012 [retrieved Jul. 15, 2016], Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation, pp. 33-42, Retrieved from the Internet: <<http://dl.acm.org/citation.cfm?id=2422363>>.

* cited by examiner

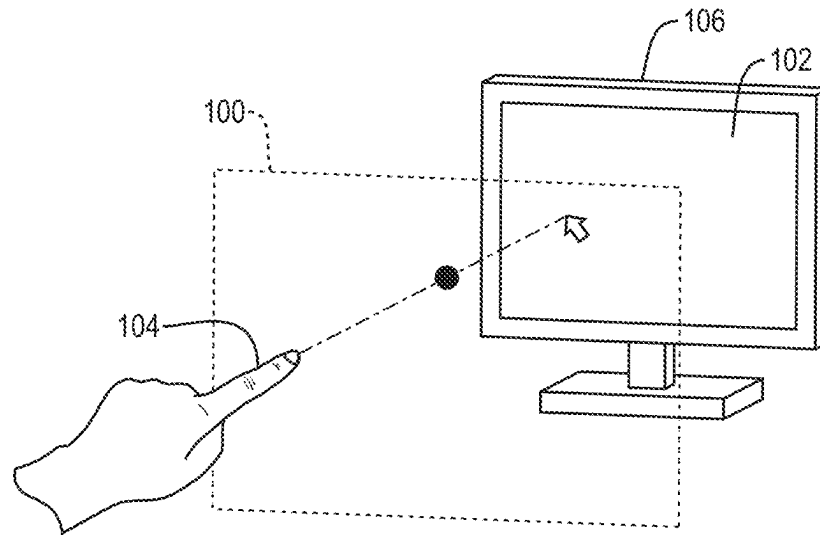


FIG. 1A

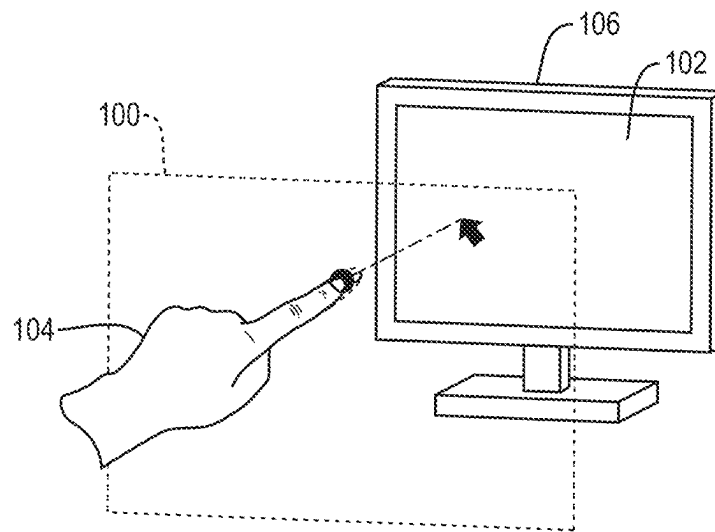


FIG. 1B

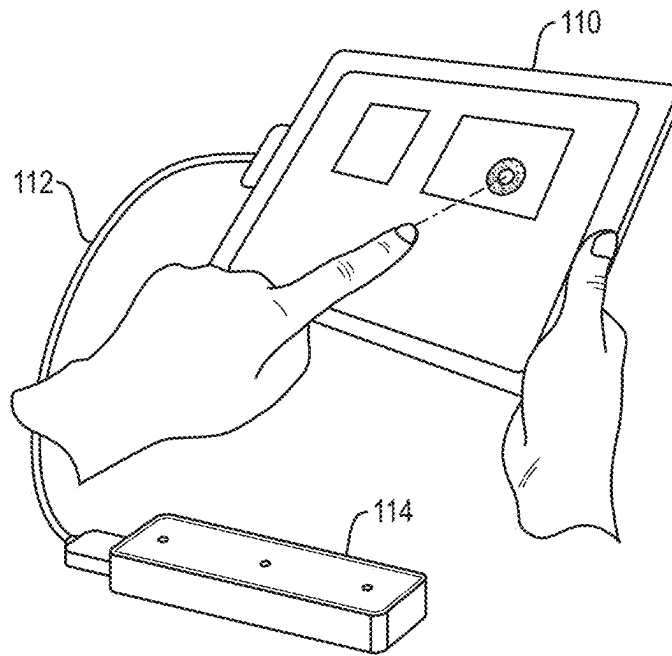


FIG. 1C-1

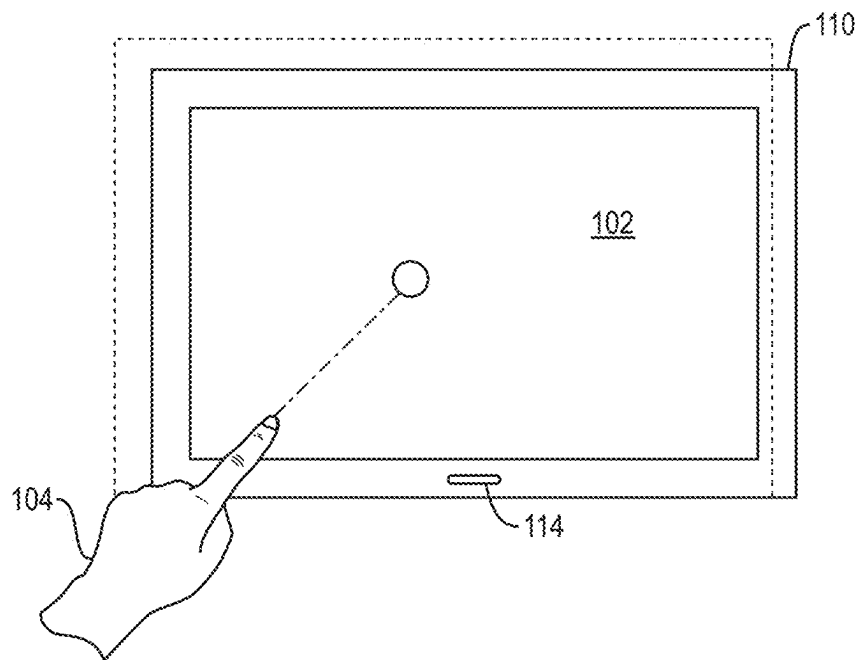


FIG. 1C-2

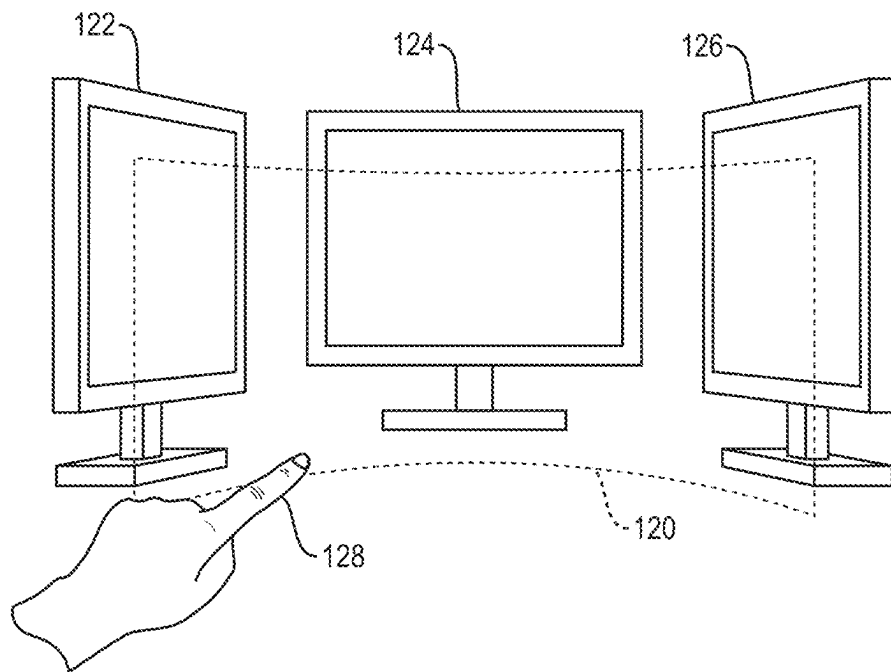


FIG. 1D

FIG. 2

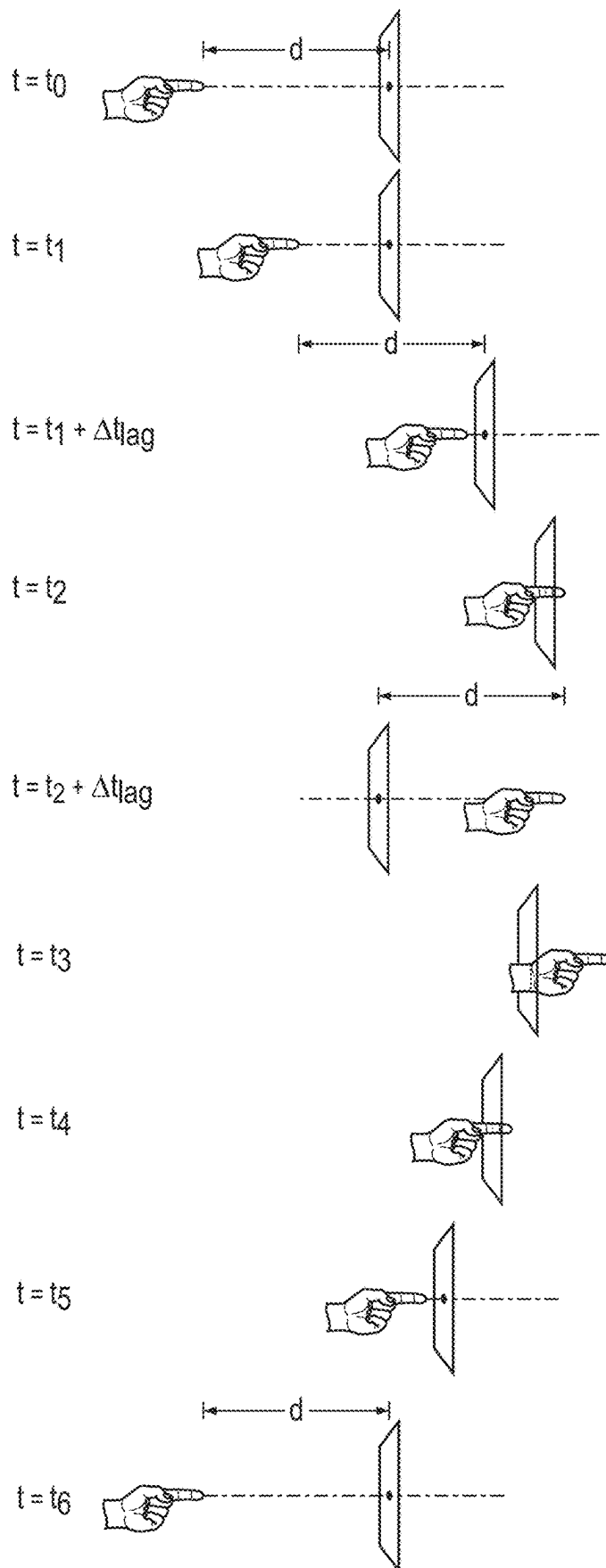


FIG. 3A

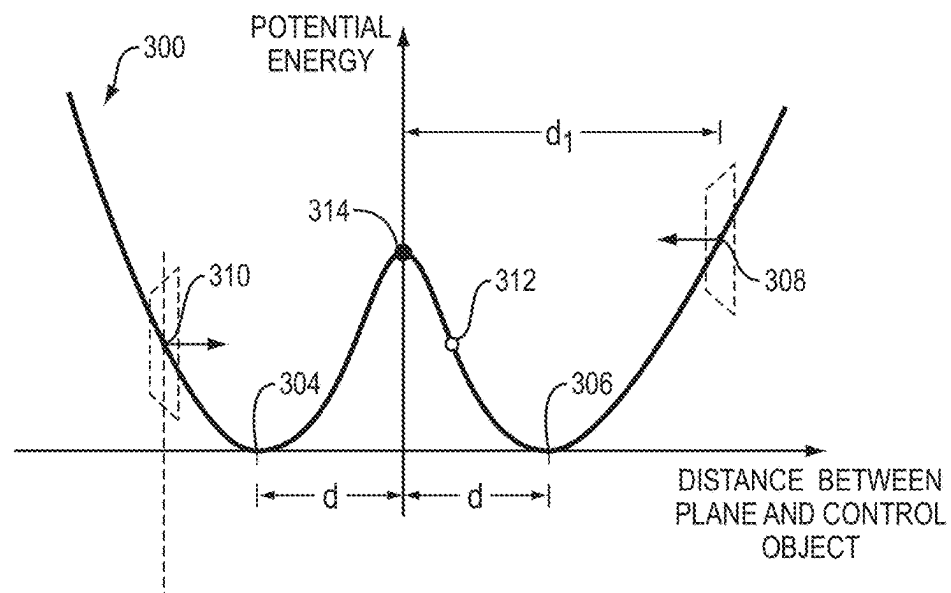
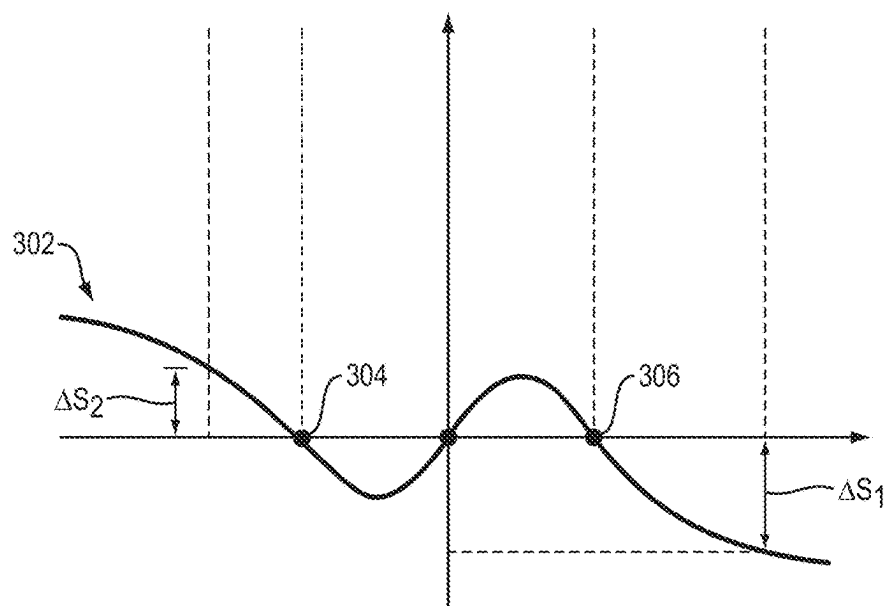


FIG. 3B



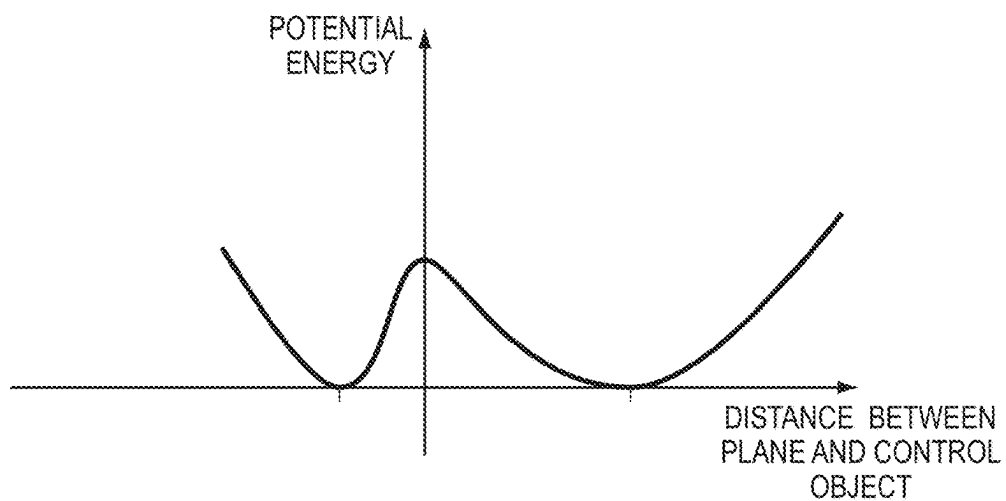


FIG. 3C

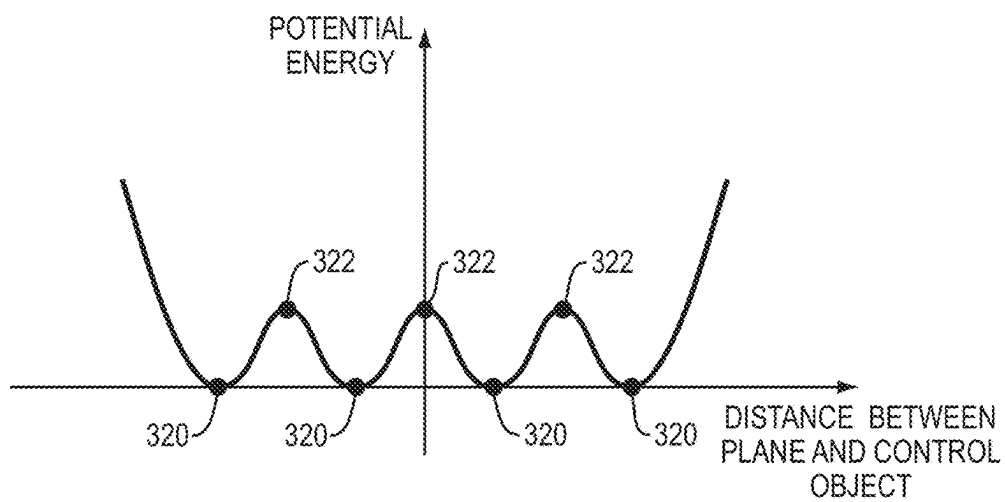


FIG. 3D

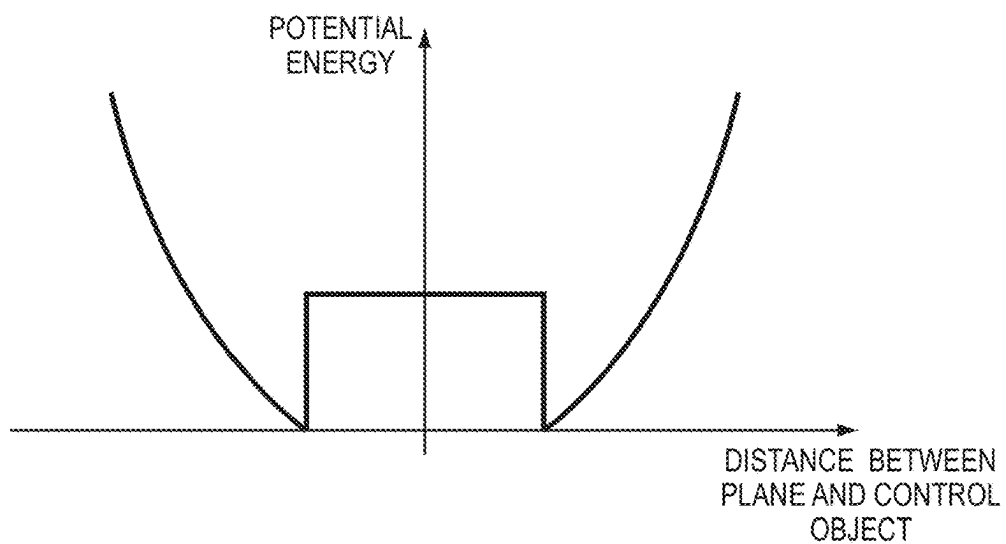


FIG. 3E

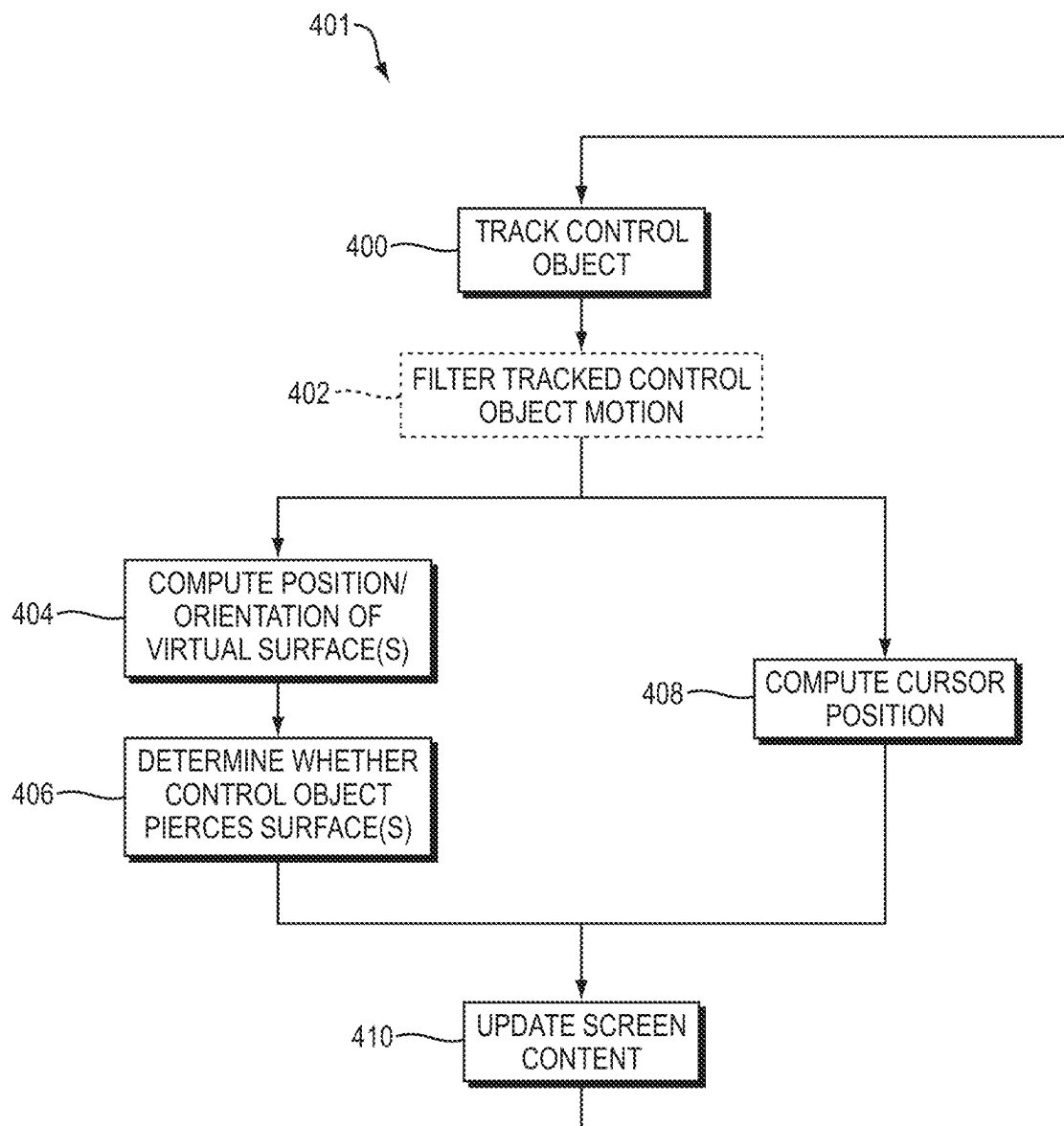


FIG. 4A

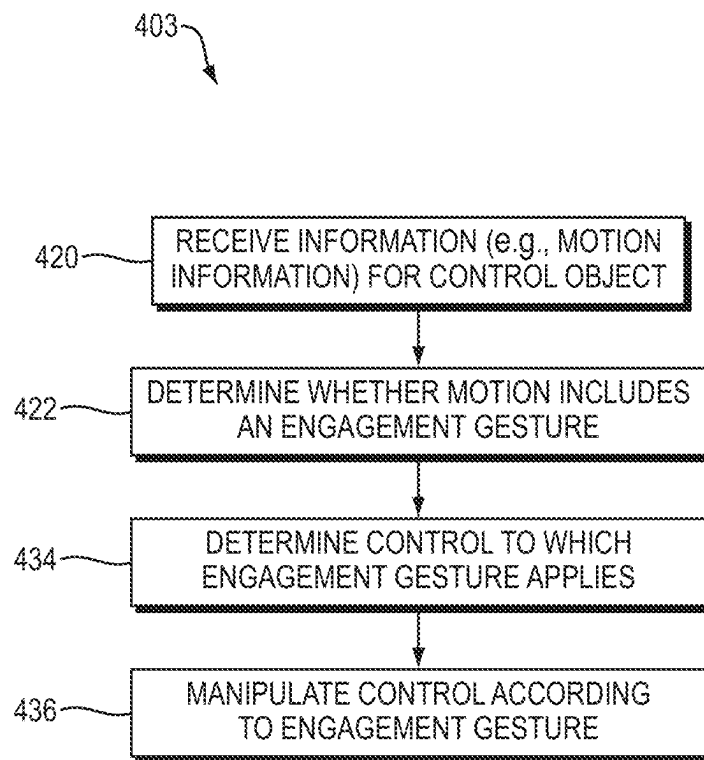


FIG. 4B

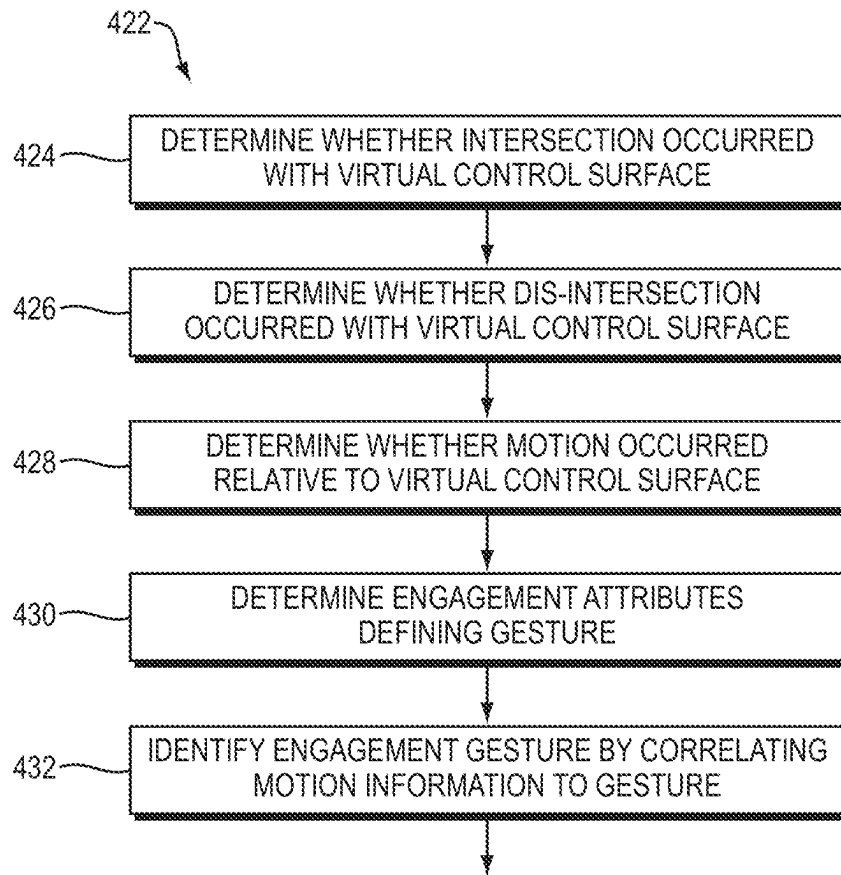


FIG. 4B-1

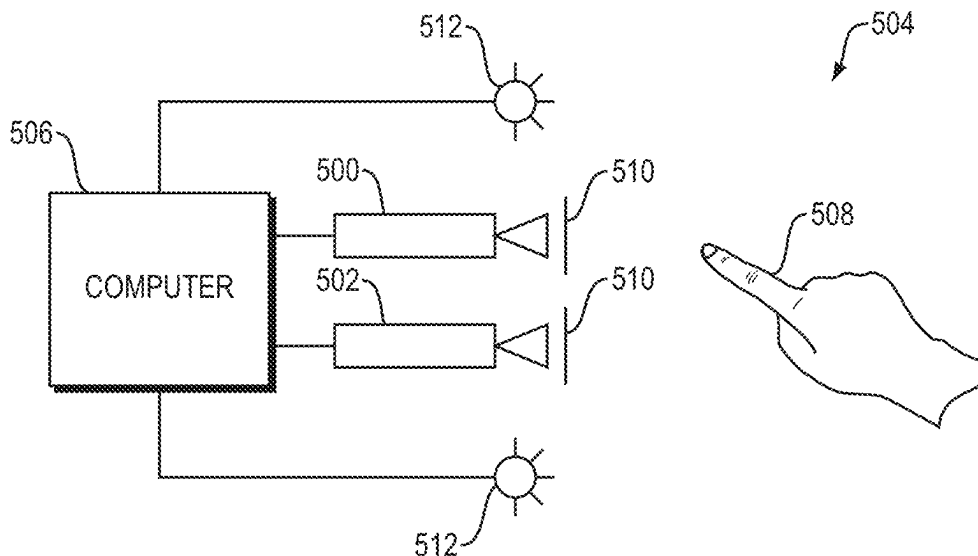


FIG. 5A

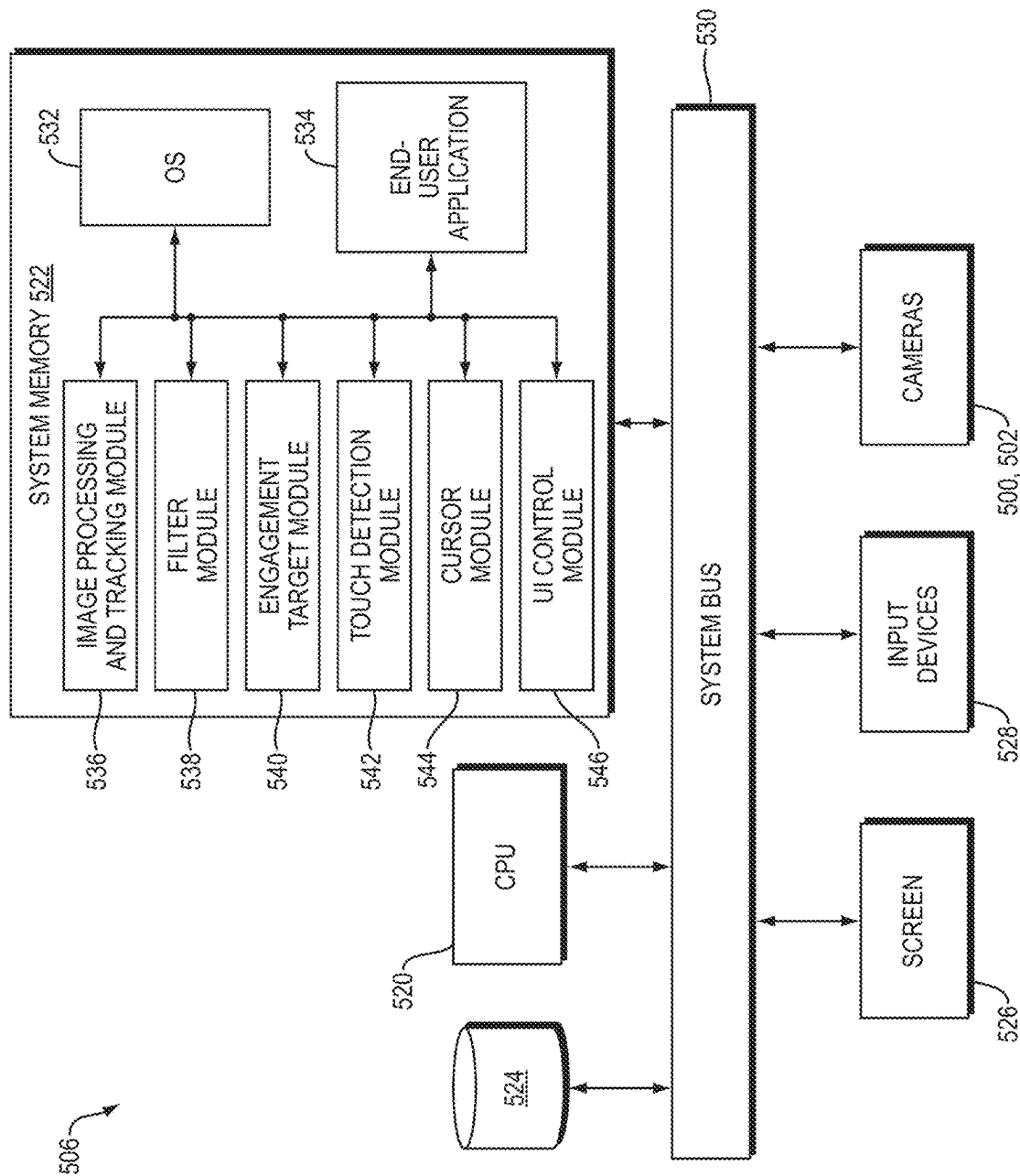


FIG. 5B

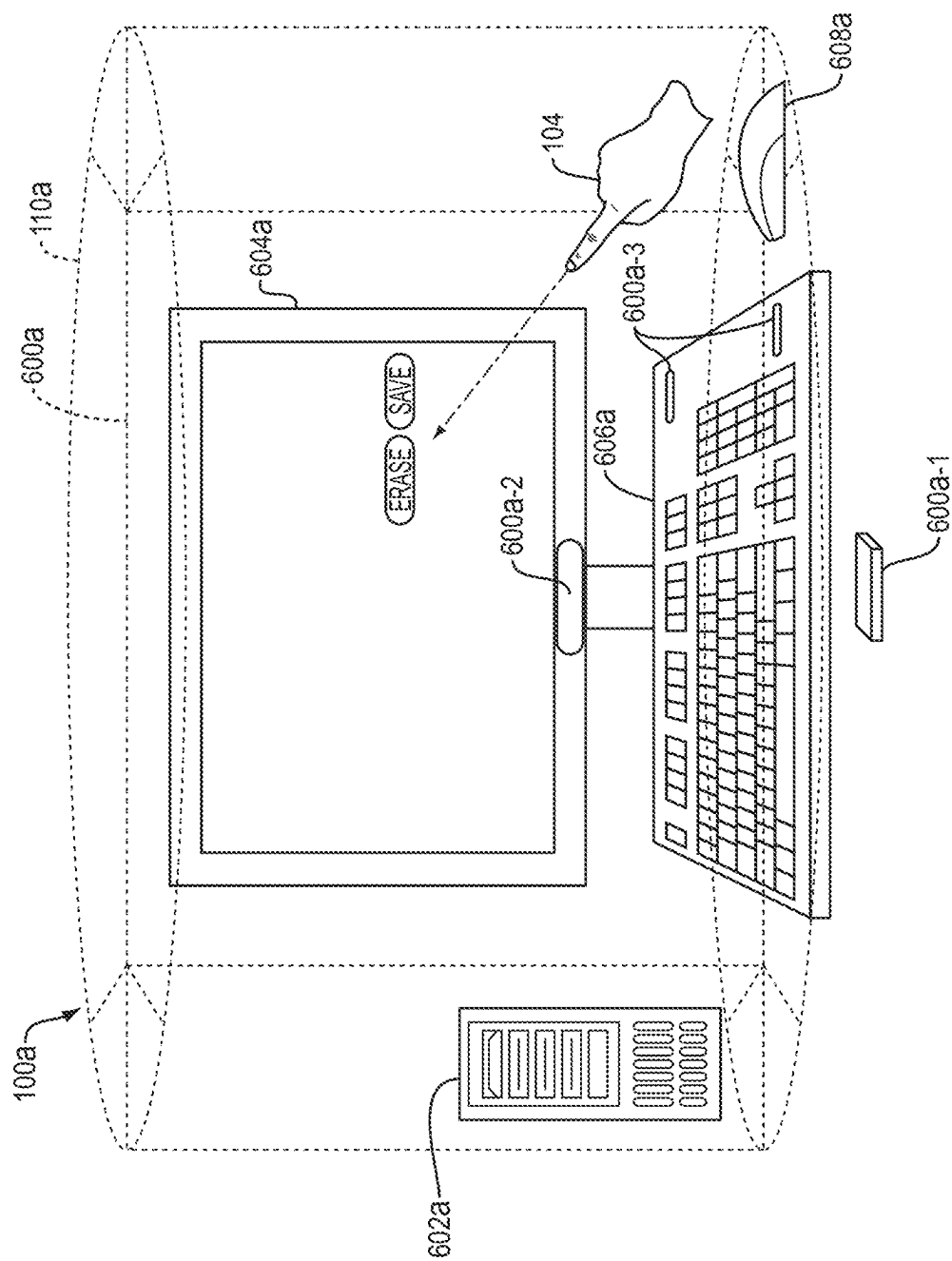


FIG. 6A

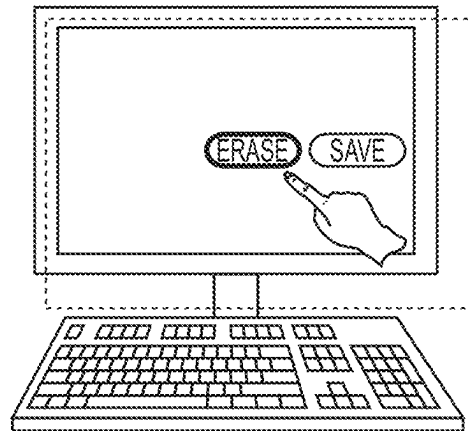


FIG. 6B

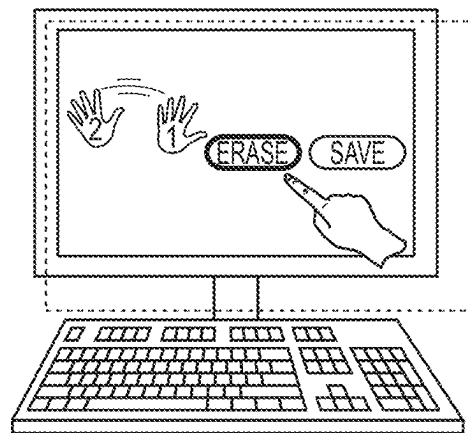


FIG. 6C

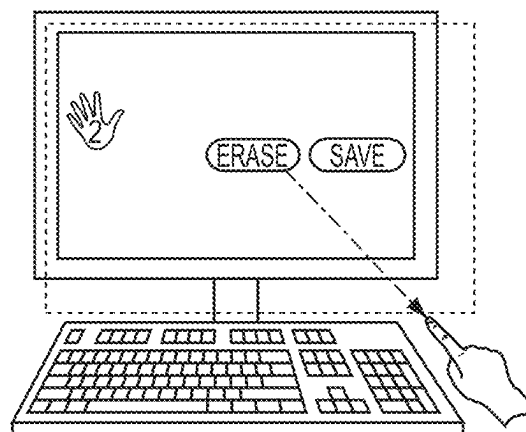
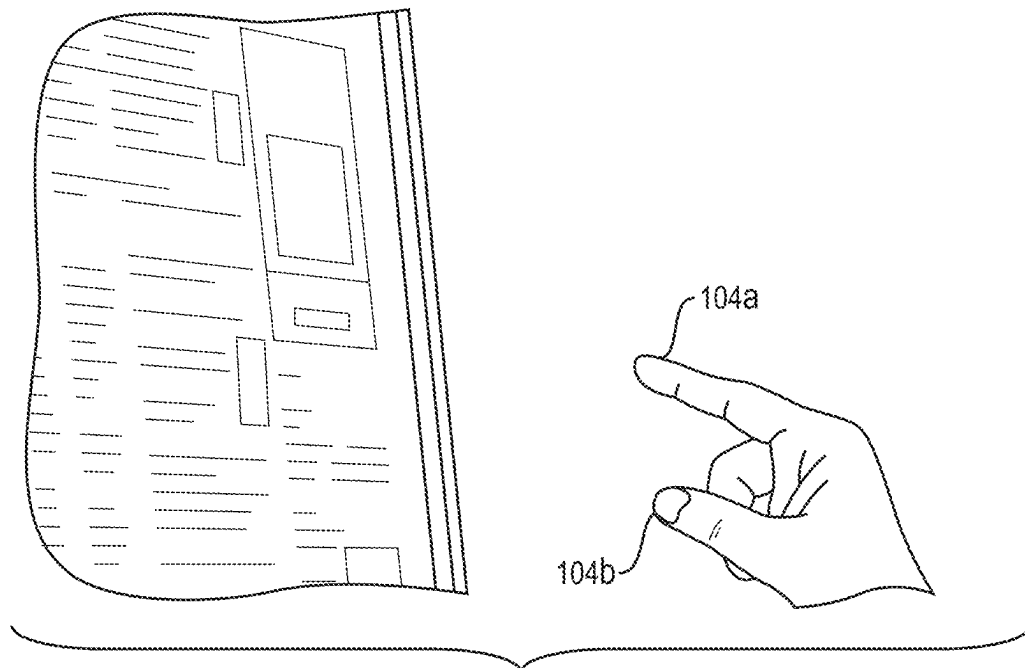
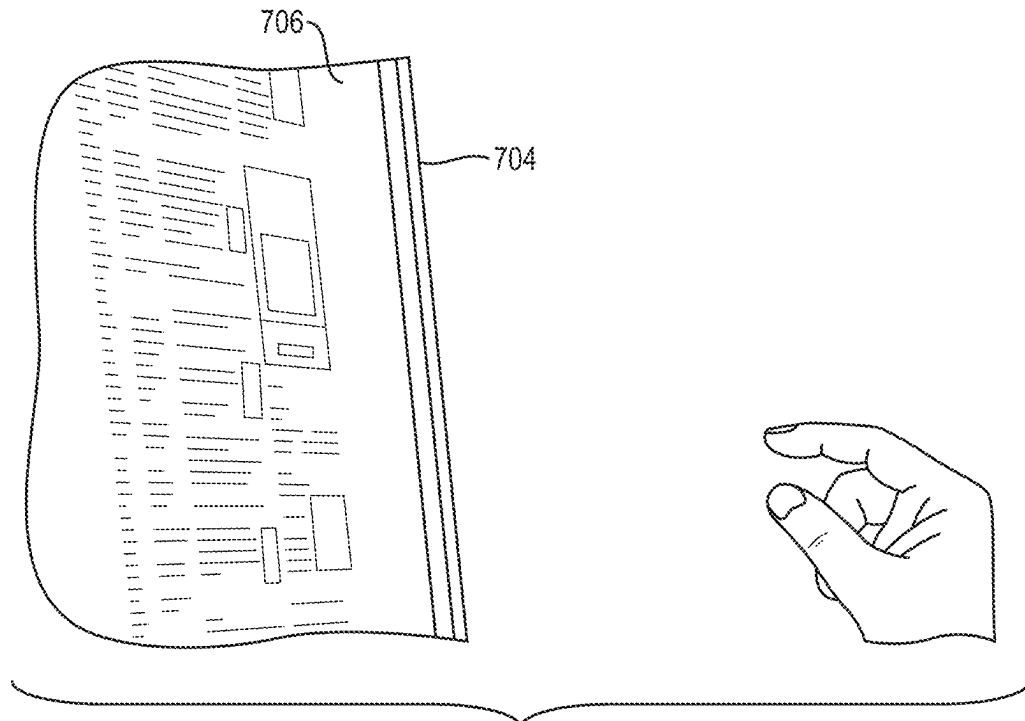


FIG. 6D



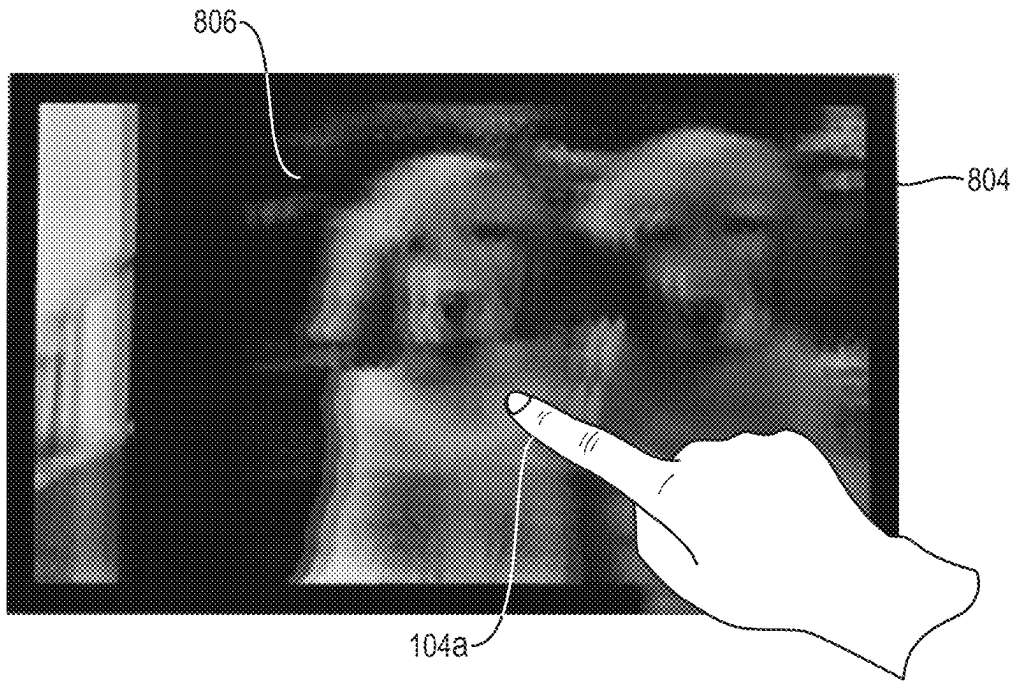


FIG. 8A

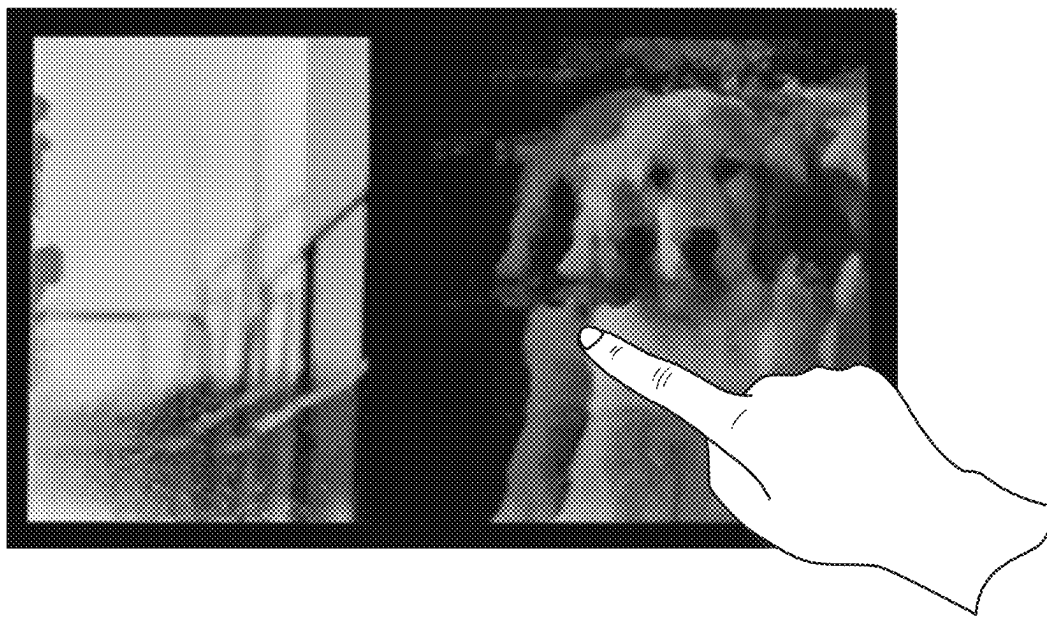


FIG. 8B

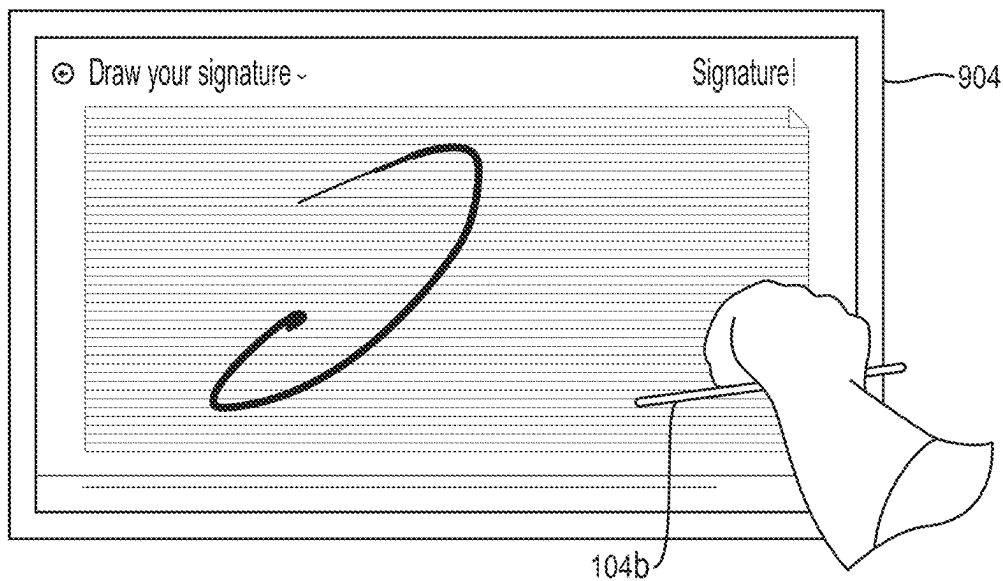


FIG. 9A

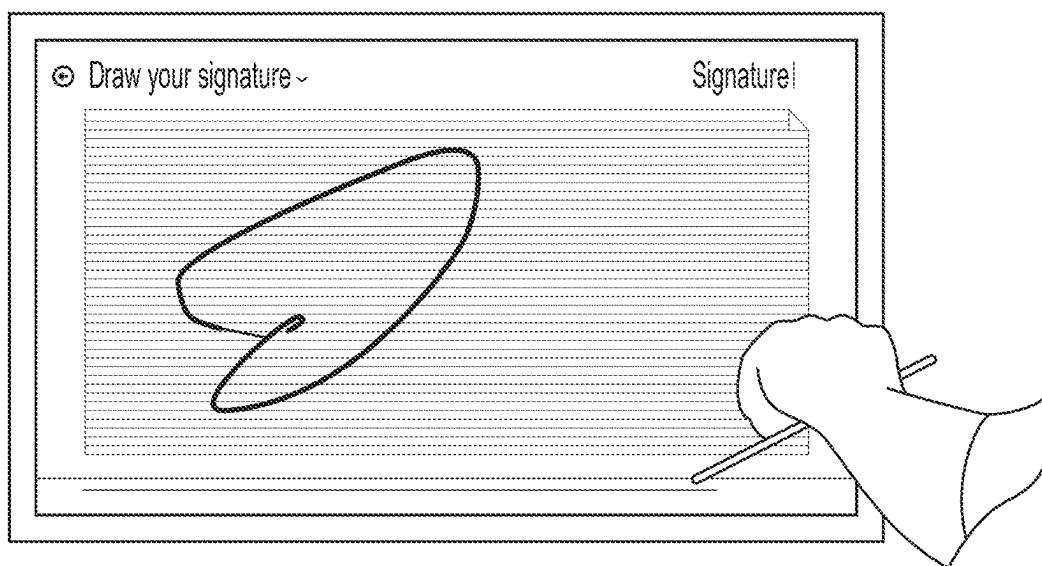


FIG. 9B

1

FREE-SPACE USER INTERFACE AND CONTROL USING VIRTUAL CONSTRUCTS

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation of U.S. patent application Ser. No. 17/833,556, filed Jun. 6, 2022, entitled “Free-space User Interface and Control Using Virtual Constructs”, which is a continuation of U.S. patent application Ser. No. 16/987,289, filed Aug. 6, 2020, entitled “Free-space User Interface and Control Using Virtual Constructs”, now U.S. Pat. No. 11,353,962 which is a continuation of U.S. patent application Ser. No. 16/054,891, filed Aug. 3, 2018, entitled “Free-space User Interface and Control Using Virtual Constructs”, now U.S. Pat. No. 10,739,862, which is a continuation of U.S. patent application Ser. No. 15/358,104, filed on Nov. 21, 2016, entitled “Free-space User Interface and Control Using Virtual Constructs”, now U.S. Pat. No. 10,042,430, which is a continuation of U.S. patent application Ser. No. 14/154,730, filed on Jan. 14, 2014, entitled “Free-space User Interface and Control Using Virtual Constructs”, now U.S. Pat. No. 9,501,152, which claims priority to and the benefit of, and incorporates herein by reference in their entireties, U.S. Provisional Application Nos. 61/825,515 and 61/825,480, both filed on May 20, 2013; Ser. No. 61/873,351, filed on Sep. 3, 2013; Ser. No. 61/877,641, filed on Sep. 13, 2013; Ser. No. 61/816,487, filed on Apr. 26, 2013; Ser. No. 61/824,691, filed on May 17, 2013; Ser. Nos. 61/752,725, 61/752,731, and 61/752,733, all filed on Jan. 15, 2013; Ser. No. 61/791,204, filed on Mar. 15, 2013; Ser. Nos. 61/808,959 and 61/808,984, both filed on Apr. 5, 2013; and No. 61/872,538, filed on Aug. 30, 2013.

TECHNICAL FIELD

Implementations relate generally to machine-user interfaces, and more specifically to the interpretation of free-space user movements as control inputs.

BACKGROUND

Current computer systems typically include a graphic user interface that can be navigated by a cursor, i.e., a graphic element displayed on the screen and movable relative to other screen content, and which serves to indicate a position on the screen. The cursor is usually controlled by the user via a computer mouse or touch pad. In some systems, the screen itself doubles as an input device, allowing the user to select and manipulate graphic user interface components by touching the screen where they are located. While touch can be convenient and relatively intuitive for many users, touch is not that accurate. Fingers are fat. The user’s fingers can easily cover multiple links on a crowded display leading to erroneous selection. Touch is also unforgiving—it requires the user’s motions to be confined to specific areas of space. For example, move one’s hand merely one key-width to the right or left and type. Nonsense appears on the screen.

Mice, touch pads, and touch screens can be cumbersome and inconvenient to use. Touch pads and touch screens require the user to be in close physical proximity to the pad (which is often integrated into a keyboard) or screen so as to be able to reach them, which significantly restricts users’ range of motion while providing input to the system. Touch is, moreover, not always reliably detected, sometimes necessitating repeated motions across the pad or screen to effect the input. Mice facilitate user input at some distance from

2

the computer and screen (determined by the length of the connection cable or the range of the wireless connection between computer and mouse), but require a flat surface with suitable surface properties, or even a special mouse pad, to function properly. Furthermore, prolonged use of a mouse, in particular if it is positioned sub-optimally relative to the user, can result in discomfort or even pain.

Accordingly, alternative input mechanisms that provide users with the advantages of touch based controls but free the user from the many disadvantages of touch based control are highly desirable.

SUMMARY

Aspects of the system and methods, described herein provide for improved machine interface and/or control by interpreting the motions (and/or position, configuration) of one or more control objects or portions thereof relative to one or more virtual control constructs defined (e.g., programmatically) in free space disposed at least partially within a field of view of an image-capture device. In implementations, the position, orientation, and/or motion of control object(s) (e.g., a user’s finger(s), thumb, etc.; a suitable hand-held pointing device such as a stylus, wand, or some other control object; portions and/or combinations thereof) are tracked relative to virtual control surface(s) to facilitate determining whether an engagement gesture has occurred. Engagement gestures can include engaging with a control (e.g., selecting a button or switch), disengaging with a control (e.g., releasing a button or switch), motions that do not involve engagement with any control (e.g., motion that is tracked by the system, possibly followed by a cursor, and/or a single object in an application or the like), environmental interactions (i.e., gestures to direct an environment rather than a specific control, such as scroll up/down), special-purpose gestures (e.g., brighten/darken screen, volume control, etc.), as well as others or combinations thereof.

Engagement gestures can be mapped to one or more controls, or a control-less screen location, of a display device associated with the machine under control. Implementations provide for mapping of movements in three-dimensional (3D) space conveying control and/or other information to zero, one, or more controls. Controls can include imbedded controls (e.g., sliders, buttons, and other control objects in an application), or environmental-level controls (e.g., windowing controls, scrolls within a window, and other controls affecting the control environment). In implementations, controls can be displayable using two-dimensional (2D) presentations (e.g., a traditional cursor symbol, cross-hairs, icon, graphical representation of the control object, or other displayable object) on, e.g., one or more display screens, and/or 3D presentations using holography, projectors, or other mechanisms for creating 3D presentations. Presentations can also be audible (e.g., mapped to sounds, or other mechanisms for conveying audible information) and/or haptic.

In an implementation, determining whether motion information defines an engagement gesture can include finding an intersection (also referred to as a contact, pierce, or a “virtual touch”) of motion of a control object with a virtual control surface, whether actually detected or determined to be imminent; dis-intersection (also referred to as a “pull back” or “withdrawal”) of the control object with a virtual control surface; a non-intersection—i.e., motion relative to a virtual control surface (e.g., wave of a hand approximately parallel to the virtual surface to “erase” a virtual chalk board); or other types of identified motions relative to the virtual

control surface suited to defining gestures conveying information to the machine. In an implementation and by way of example, one or more virtual control constructs can be defined computationally (e.g., programmatically using a computer or other intelligent machinery) based upon one or more geometric constructs to facilitate determining occurrence of engagement gestures from information about one or more control objects (e.g., hand, tool, combinations thereof) captured using imaging systems, scanning systems, or combinations thereof. Virtual control constructs in an implementation can include virtual surface constructs, virtual linear or curvilinear constructs, virtual point constructs, virtual solid constructs, and complex virtual constructs comprising combinations thereof. Virtual surface constructs can comprise one or more surfaces, e.g., a plane, curved open surface, closed surface, bounded open surface, or generally any multi-dimensional virtual surface definable in two or three dimensions. Virtual linear or curvilinear constructs can comprise any one-dimensional virtual line, curve, line segment or curve segment definable in one, two, or three dimensions. Virtual point constructs can comprise any zero-dimensional virtual point definable in one, two, or three dimensions. Virtual solids can comprise one or more solids, e.g., spheres, cylinders, cubes, or generally any three-dimensional virtual solid definable in three dimensions.

In an implementation, an engagement target can be defined using one or more virtual construct(s) coupled with a virtual control (e.g., slider, button, rotatable knob, or any graphical user interface component) for presentation to user(s) by a presentation system (e.g., displays, 3D projections, holographic presentation devices, non-visual presentation systems such as haptics, audio, and the like, any other devices for presenting information to users, or combinations thereof). Coupling a virtual control with a virtual construct enables the control object to “aim” for, or move relative to, the virtual control—and therefore the virtual control construct. Engagement targets in an implementation can include engagement volumes, engagement surfaces, engagement lines, engagement points, or the like, as well as complex engagement targets comprising combinations thereof. An engagement target can be associated with an application or non-application (e.g., OS, systems software, etc.) so that virtual control managers (i.e., program routines, classes, objects, etc. that manage the virtual control) can trigger differences in interpretation of engagement gestures including presence, position and/or shape of control objects, control object motions, or combinations thereof to conduct machine control. As explained in more detail below with reference to example implementations, engagement targets can be used to determine engagement gestures by providing the capability to discriminate between engagement and non-engagement (e.g., virtual touches, moves in relation to, and/or virtual pierces) of the engagement target by the control object.

In an implementation, determining whether motion information defines an engagement gesture can include determining one or more engagement attributes from the motion information about the control object. In an implementation, engagement attributes include motion attributes (e.g., speed, acceleration, duration, distance, etc.), gesture attributes (e.g., hand, two hands, tools, type, precision, etc.), other attributes and/or combinations thereof.

In an implementation, determining whether motion information defines an engagement gesture can include filtering motion information to determine whether motion comprises an engagement gesture. Filtering can be applied based upon engagement attributes, characteristics of motion, position in

space, other criteria, and/or combinations thereof. Filtering can enable identification of engagement gestures, discrimination of engagement gestures from extraneous motions, discrimination of engagement gestures of differing types or meanings, and so forth.

In an implementation, sensing an engagement gesture provides an indication for selecting a mode to control a user interface of the machine (e.g., an “engaged mode” simulating a touch, or a “disengaged mode” simulating no contact and/or a hover in which a control is selected but not actuated). Other modes useful in various implementations include an “idle,” in which no control is selected nor virtually touched, and a “lock,” in which the last control to be engaged with remains engaged until disengaged. Yet further, hybrid modes can be created from the definitions of the foregoing modes in implementations.

In various implementations, to trigger an engaged mode—corresponding to, e.g., touching an object or a virtual object displayed on a screen—the control object’s motion toward an engagement target such as a virtual surface construct (i.e., a plane, plane portion, or other (non-planar or curved) surface computationally or programmatically defined in space, but not necessarily corresponding to any physical surface) can be tracked; the motion can be, e.g., a forward motion starting from a disengaged mode, or a backward retreating motion. When the control object reaches a spatial location corresponding to this virtual surface construct—i.e., when the control object intersects “touches” or “pierces” the virtual surface construct—the user interface (or a component thereof, such as a cursor, user-interface control, or user-interface environment) is operated in the engaged mode; as the control object retracts from the virtual surface construct, user-interface operation switches back to the disengaged mode.

In implementations, the virtual surface construct can be fixed in space, e.g., relative to the screen; for example, it can be defined as a plane (or portion of a plane) parallel to and located several inches in front of the screen in one application, or as a curved surface defined in free space convenient to one or more users and optionally proximately to display(s) associated with one or more machines under control. The user can engage this plane while remaining at a comfortable distance from the screen (e.g., without needing to lean forward to reach the screen). The position of the plane can be adjusted by the user from time to time. In implementations, however, the user is relieved of the need to explicitly change the plane’s position; instead, the plane (or other virtual surface construct) automatically moves along with, as if tethered to, the user’s control object. For example, a virtual plane can be computationally defined as perpendicular to the orientation of the control object and located a certain distance, e.g., 3-4 millimeters, in front of its tip when the control object is at rest or moving with constant velocity. As the control object moves, the plane follows it, but with a certain time lag (e.g., 0.2 second). As a result, as the control object accelerates, the distance between its tip and the virtual touch plane changes, allowing the control object, when moving towards the plane, to eventually “catch” the plane—that is, the tip of the control object to touch or pierce the plane. Alternatively, instead of being based on a fixed time lag, updates to the position of the virtual plane can be computed based on a virtual energy potential defined to accelerate the plane towards (or away from) the control object tip depending on the plane-to-tip distance, likewise allowing the control object to touch or pierce the plane. Either way, such virtual touching or piercing can be interpreted as engagement events. Further, in some implemen-

5

tations, the degree of piercing (i.e., the distance beyond the plane that the control object reaches) is interpreted as an intensity level. To guide the user as she engages with or disengages from the virtual plane (or other virtual surface construct), the cursor symbol can encode the distance from the virtual surface visually, e.g., by changing in size with varying distance.

In an implementation, once engaged, further movements of the control object can serve to move graphical components across the screen (e.g., drag an icon, shift a scroll bar, etc.), change perceived “depth” of the object to the viewer (e.g., resize and/or change shape of objects displayed on the screen in connection, alone, or coupled with other visual effects) to create perception of “pulling” objects into the foreground of the display or “pushing” objects into the background of the display, create new screen content (e.g., draw a line), or otherwise manipulate screen content until the control object disengages (e.g., by pulling away from the virtual surface, indicating disengagement with some other gesture of the control object (e.g., curling the forefinger backward); and/or with some other movement of a second control object (e.g., waving the other hand, etc.)). Advantageously, tying the virtual surface construct to the control object (e.g., the user’s finger), rather than fixing it relative to the screen or other stationary objects, allows the user to consistently use the same motions and gestures to engage and manipulate screen content regardless of his precise location relative to the screen. To eliminate the inevitable jitter typically accompanying the control object’s movements and which might otherwise result in switching back and forth between the modes unintentionally, the control object’s movements can be filtered and the cursor position thereby stabilized. Since faster movements will generally result in more jitter, the strength of the filter can depend on the speed of motion.

Accordingly, in one aspect, a computer-implemented method of controlling a machine user interface is provided. The method involves receiving information including motion information for a control object; determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to at least one virtual control construct defined within a field of view of an image capturing device; determining a control to which the engagement gesture is applicable; and manipulating the control according to at least the motion information. The method can further include updating at least a spatial position of the virtual control construct(s) based at least in part on a spatial position of the control object determined from the motion information, thereby enabling the spatial position of the virtual control construct(s) to follow tracked motions of the control object.

In some implementations, determining whether a motion of the control object is an engagement gesture includes determining whether an intersection between the control object and the virtual control construct(s), a dis-intersection of the control object from the virtual control construct(s), or a motion of the control object relative to the virtual control construct(s) occurred. The method can further include determining from the motion information whether the engagement includes continued motion after intersection. In some implementations, determining from the motion information whether a motion of the control object is an engagement gesture includes determining from the motion information one or more engagement attributes (e.g., a potential energy) defining an engagement gesture. In some implementations, determining whether a motion of the control object is an

6

engagement gesture includes identifying an engagement gesture by correlating motion information to at least one engagement gesture based at least upon one or more of motion of the control object, occurrence of any of an intersection, a dis-intersection or a non-intersection of the control object with the virtual control construct, and the set of engagement attributes.

Determining a control to which the engagement gesture is applicable can include selecting a control associated with an application, a control associated with an operating environment, and/or a special control. Manipulating a control according to at least the motion information can include controlling a user interface in a first mode, and otherwise controlling the user interface in a second mode different from the first mode.

In another aspect, a computer-implemented method of controlling a machine user interface is provided. The method includes receiving information including motion information for a control object. Further, it includes determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to at least one virtual control construct defined within a field of view of an image capturing device by (i) determining whether an intersection occurred between control object and at least one virtual control construct, and when an intersection has occurred determining from the motion information whether the engagement includes continued motion after intersection; otherwise (ii) determining whether a dis-intersection of the control object from the at least one virtual control construct occurred; otherwise (iii) determining whether motion of the control object occurred relative to at least one virtual control construct; (iv) determining from the motion information a set of engagement attributes defining an engagement gesture; and (v) identifying an engagement gesture by correlating motion information to at least one engagement gesture based at least upon one or more of motion of the control object, occurrence of any of an intersection, a dis-intersection or a non-intersection of the control object with the virtual control construct, and the set of engagement attributes. Further, the method involves determining a control to which the engagement gesture is applicable, and manipulating the control according to at least the engagement gesture.

In another aspect, a computer-implemented method for facilitating control of a user interface via free-space motions of a control object is provided. One method implementation includes receiving data indicative of tracked motions of the control object, and computationally (i.e., using a processor) defining a virtual control construct and updating a spatial position (and, in some implementations, also a spatial orientation) of the virtual control construct based at least in part on the data such that the position of the virtual control construct follows the tracked motions of the control object. Further, implementations of the method involve computationally determining whether the control object intersects the virtual control construct, and, if so, controlling the user interface in a first mode (e.g., an engaged mode), and otherwise controlling the user interface in a second mode different from the first mode (e.g., a disengaged mode).

In some implementations, the virtual control construct follows the tracked motions of the control object with a time lag, which can be fixed or, e.g., depend on a motion parameter of the control object. In alternative implementations, the spatial position of the virtual control construct is updated based on a current distance between the control object and the virtual control construct, e.g., in accordance

with a virtual energy potential defined as a function of that distance. The virtual energy potential can have minima at steady-state distances between the control object and the virtual control construct in the engaged mode and the disengaged mode. In some implementations, the steady-state distance in the engaged mode is equal to the steady-state distance in the disengaged mode; in other implementations, the steady-state distance in the engaged mode is larger (or smaller) than the steady-state distance in the disengaged mode.

Determining whether the control object intersects the virtual control construct can involve computing an intersection of a straight line through the axis of the control object with a screen displaying the user interface or, alternatively, computationally projecting a tip of the control object perpendicularly onto the screen. Controlling the user interface can involve updating the screen content based, at least in part, on the tracked control object motions and the operational mode (e.g., the engaged or disengaged mode). For example, in some implementations, it involves operating a cursor variably associated with a screen position; a cursor symbol can be displayed on the screen at that position. The cursor can also be indicative of a distance between the control object and the virtual control construct. (The term "cursor," as used herein, refers to a control element operable to select a screen position—whether or not the control element is actually displayed—and manipulate screen content via movement across the screen, i.e., changes in the selected position.) In some implementations, the method further includes computationally determining, for a transition from the disengaged mode to the engaged mode, a degree of penetration of the virtual control construct by the control object, and controlling the user interface based at least in part thereon.

The method can also include acquiring a temporal sequence of images of the control object (e.g., with a camera system having depth-sensing capability) and/or computationally tracking the motions of the control object based on the sequence of images. In some implementations, the control object motions are computationally filtered based, at least in part, on the control object's velocity.

In another aspect, implementations pertain to a computer-implemented method for controlling a user interface via free-space motions of a control object. The method involves receiving motion information indicating positions of a control object being tracked in free space, and, using a processor, (i) defining a virtual control construct, at least a portion thereof having a spatial position determined based at least in part on the motion information such that the virtual control construct portion is positioned proximate to the control object, (ii) determining from the motion information whether the tracked motions of the control object indicate that the control object has intersected the virtual control construct, and (iii) switching from conducting control of a user interface in a first mode to conducting control of the user interface in a second mode based at least in part upon an occurrence of the control object intersecting the virtual control construct. The method can further involve updating at least the spatial position of the virtual control construct portion based at least in part on the motion information such that the virtual control construct portion is enabled to follow the control object.

In another aspect, implementations provide a system for controlling a machine user interface via free-space motions of a control object tracked with an image capturing device, the system including a processor and memory. The memory stores (i) motion information for the control object; and (ii)

processor-executable instructions for causing the processor to determine from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to at least one virtual control construct defined within a field of view of the image capturing device, to determine a control to which the engagement gesture is applicable, and to manipulate the control according to at least the motion information.

Yet another aspect pertains to a non-transitory machine-readable medium. In implementations, the medium stores one or more instructions which, when executed by one or more processors, cause the one or more processors to determine from motion information received for a control object whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to at least one virtual control construct defined within a field of view of an image capturing device; determine a control to which the engagement gesture is applicable; and manipulate the control according to at least the motion information.

In a further aspect, a system for controlling a user interface via free-space motions of a control object tracked by a motion-capture system is provided. The system includes a processor and associated memory, the memory storing processor-executable instructions for causing the processor to (i) computationally define a virtual control construct relative to the control object and update at least a spatial position thereof, based at least in part on the tracked motions of the control object, such that the spatial position of the virtual control construct follows the tracked motions of the control object, (ii) computationally determine whether the control object, in the current spatial position, intersects the virtual control construct, and (iii) if so, control the user interface in a first mode, and otherwise control the user interface in a second mode different from the first mode. In some implementations, the first and second modes are engaged and disengaged modes, respectively. Execution of the instructions by the processor can cause the processor to compute a position of the virtual control construct relative to the current position of the control object such that the virtual control construct follows the tracked motions of the control object with a time lag, and/or to update the spatial position of the virtual control construct in accordance with a virtual energy potential defined as a function of a distance between the control object and the virtual control construct.

The system can further include the motion-capture system for tracking the motions of the control object in three dimensions based on a temporal sequence of images of the control object. In some implementations, the motion-capture system includes one or more camera(s) acquiring the images and a plurality of image buffers for storing a most recent set of the images. The system can also have a filter for computationally filtering the motions of the control object based, at least in part, on a velocity of these motions. In addition, the system can include a screen for displaying the user interface; execution of the instructions by the processor can cause the processor to update screen content based, at least in part, on the mode and the tracked motions of the control object. In some implementation, execution of the instructions by the processor causes the processor to operate a cursor associated with a position on a screen based, at least in part, on the mode and the tracked motions of the control object. The screen can display a cursor symbol at the associated position; the cursor symbol can be indicative of a distance between the control object and the virtual control construct.

In another aspect, a non-transitory machine-readable medium storing one or more instructions is provided in which, when executed by one or more processors, cause the one or more processors to (i) computationally define a virtual control construct and update at least a spatial position thereof based at least in part on data indicative of tracked motions of a control object such that the position of the virtual control construct follows the tracked motions of the control object, (ii) computationally determine whether the control object intersects the virtual control construct, and (iii) if so, control the user interface in a first mode, and otherwise control the user interface in a second mode different from the first mode.

In yet another aspect, a computer-implemented method for facilitating control of a user interface via free-space motions of a control object is provided. The method involves receiving data indicative of tracked motions of the control object, and, using a processor, (i) computationally defining a virtual control construct and updating at least a spatial position thereof based at least in part on the data such that the position of the virtual control construct follows the tracked motions of the control object, (ii) computationally detecting when a tip of the control object transitions from one side of the virtual control construct to another side, and (iii) whenever it does, switching between two modes of controlling the user interface.

In a further aspect, yet another computer-implemented method for facilitating control of a user interface via free-space motions of a control object is provided. The method includes tracking motions of a control object and a gesturer; using a processor to continuously determine computationally whether the control object intersects a virtual control construct located at a temporarily fixed location in space and, if so, controlling the user interface in a first mode and otherwise controlling the user interface in a second mode different from the first mode; and, each time upon recognition of a specified gesture performed by the gesturer, using the processor to relocate the virtual control construct to a specified distance from an instantaneous position of the control object.

Among other aspects, implementations can enable quicker, crisper gesture based or "free space" (i.e., not requiring physical contact) interfacing with a variety of machines (e.g., a computing systems, including desktop, laptop, tablet computing devices, special purpose computing machinery, including graphics processors, embedded micro-controllers, gaming consoles, audio mixers, or the like; wired or wirelessly coupled networks of one or more of the foregoing, and/or combinations thereof), obviating or reducing the need for contact-based input devices such as a mouse, joystick, touch pad, or touch screen.

BRIEF DESCRIPTION OF THE DRAWINGS

The foregoing will be more readily understood from the following detailed description, in particular, when taken in conjunction with the drawings, in which:

FIGS. 1A and 1B are perspective views of a planar virtual surface construct and a control object in the disengaged and engaged modes, respectively, illustrating free-space gesture control of a desktop computer in accordance with various implementations;

FIG. 1C-1 is a perspective view of a tablet connected to a motion-capture device, illustrating free-space gesture control of the tablet in accordance with various implementations;

FIG. 1C-2 is a perspective view of a tablet incorporating a motion-capture device, illustrating free-space gesture control of the tablet in accordance with various implementations;

FIG. 1D is a perspective view of a curved virtual surface construct accommodating free-space gesture control of a multi-screen computer system in accordance with various implementations;

FIG. 2 illustrates motion of a virtual surface construct relative to a user's finger in accordance with various implementations;

FIGS. 3A and 3B are plots of a virtual energy potential and its derivative, respectively, in accordance with various implementations for updating the position of a virtual surface construct;

FIGS. 3C-3E are plots of alternative virtual energy potentials in accordance with various implementations for updating the position of a virtual surface construct;

FIGS. 4A, 4B, and 4B-1 are flow charts illustrating methods for machine and/or user interface control in accordance with various implementations;

FIG. 5A is a schematic diagram of a system for tracking control object movements in accordance with various implementations;

FIG. 5B is a block diagram of a computer system for machine control based on tracked control object movements in accordance with various implementations;

FIGS. 6A-6D illustrate a free-space compound gesture in accordance with various implementations;

FIGS. 7A and 7B illustrate, in two snap shots, a zooming action performed by a user via a free-space gesture in accordance with various implementations;

FIGS. 8A and 8B illustrate, in two snap shots, a swiping action performed by a user via a free-space gesture in accordance with various implementations; and

FIGS. 9A and 9B illustrate, in two snap shots, a drawing action performed by a user via free-space hand motions in accordance with various implementations.

DETAILED DESCRIPTION

System and methods in accordance herewith generally utilize information about the motion of a control object, such as a user's finger or a stylus, in three-dimensional space to operate a user interface and/or components thereof based on the motion information. Various implementations take advantage of motion-capture technology to track the motions of the control object in real time (or near real time, i.e., sufficiently fast that any residual lag between the control object and the system's response is unnoticeable or practically insignificant). Other implementations can use synthetic motion data (e.g., generated by a computer game) or stored motion data (e.g., previously captured or generated). References to motions in "free space" or "touchless" motions are used herein with reference to an implementation to distinguish motions tied to and/or requiring physical contact of the moving object with a physical surface to effect input; however, in some applications, the control object can contact a physical surface ancillary to providing input, in such case the motion is still considered a "free-space" motion. Further, in some implementations, the virtual surface can be defined to co-reside at or very near a physical surface (e.g., a virtual touch screen can be created by defining a (substantially planar) virtual surface at or very near the screen of a display (e.g., television, monitor, or the like); or a virtual active table

top can be created by defining a (substantially planar) virtual surface at or very near a table top convenient to the machine receiving the input).

A “control object” as used herein with reference to an implementation is generally any three-dimensionally movable object or appendage with an associated position and/or orientation (e.g., the orientation of its longest axis) suitable for pointing at a certain location and/or in a certain direction. Control objects include, e.g., hands, fingers, feet, or other anatomical parts, as well as inanimate objects such as pens, styluses, handheld controls, portions thereof, and/or combinations thereof. Where a specific type of control object, such as the user’s finger, is used hereinafter for ease of illustration, it is to be understood that, unless otherwise indicated or clear from context, any other type of control object can be used as well.

A “virtual control construct” as used herein with reference to an implementation denotes a geometric locus defined (e.g., programmatically) in space and useful in conjunction with a control object, but not corresponding to a physical object; its purpose is to discriminate between different operational modes of the control object (and/or a user-interface element controlled therewith, such as a cursor) based on whether the control object intersects the virtual control construct. The virtual control construct, in turn, can be, e.g., a virtual surface construct (a plane oriented relative to a tracked orientation of the control object or an orientation of a screen displaying the user interface) or a point along a line or line segment extending from the tip of the control object.

The term “intersect” is herein used broadly with reference to an implementation to denote any instance in which the control object, which is an extended object, has at least one point in common with the virtual control construct and, in the case of an extended virtual control construct such as a line or two-dimensional surface, is not parallel thereto. This includes “touching” as an extreme case, but typically involves that portions of the control object fall on both sides of the virtual control construct.

Using the output of a suitable motion-capture system or motion information received from another source, various implementations facilitate user input via gestures and motions performed by the user’s hand or a (typically handheld) pointing device. For example, in some implementations, the user can control the position of a cursor and/or other object on the screen by pointing at the desired screen location, e.g., with his index finger, without the need to touch the screen. The position and orientation of the finger relative to the screen, as determined by the motion-capture system, can be used to compute the intersection of a straight line through the axis of the finger with the screen, and a cursor symbol (e.g., an arrow, circle, cross hair, or hand symbol) can be displayed at the point of intersection. If the range of motion causes the intersection point to move outside the boundaries of the screen, the intersection with a (virtual) plane through the screen can be used, and the cursor motions can be re-scaled, relative to the finger motions, to remain within the screen boundaries. Alternatively to extrapolating the finger towards the screen, the position of the finger (or control object) tip can be projected perpendicularly onto the screen; in this implementation, the control object orientation can be disregarded. As will be readily apparent to one of skill in the art, many other ways of mapping the control object position and/or orientation onto a screen location can, in principle, be used; a particular mapping can be selected based on considerations such as, without limitation, the requisite amount of information

about the control object, the intuitiveness of the mapping to the user, and the complexity of the computation. For example, in some implementations, the mapping is based on intersections with or projections onto a (virtual) plane defined relative to the camera, under the assumption that the screen is located within that plane (which is correct, at least approximately, if the camera is correctly aligned relative to the screen), whereas, in other implementations, the screen location relative to the camera is established via explicit calibration (e.g., based on camera images including the screen).

In some implementations, the cursor can be operated in at least two modes: a disengaged mode in which it merely indicates a position on the screen, typically without otherwise affecting the screen content; and one or more engaged modes, which allow the user to manipulate the screen content. In the engaged mode, the user can, for example, drag graphical user-interface elements (such as icons representing files or applications, controls such as scroll bars, or displayed objects) across the screen, or draw or write on a virtual canvas. Further, transient operation in the engaged mode can be interpreted as a click event. Thus, operation in the engaged mode generally corresponds to, or emulates, touching a touch screen or touch pad, or controlling a mouse with a mouse button held down.

The term “cursor,” as used in this discussion, refers generally to the cursor functionality rather than the visual element; in other words, the cursor is a control element operable to select a screen position—whether or not the control element is actually displayed—and manipulate screen content via movement across the screen, i.e., changes in the selected position. The cursor need not always be visible in the engaged mode. In some instances, a cursor symbol still appears, e.g., overlaid onto another graphical element that is moved across the screen, whereas in other instances, cursor motion is implicit in the motion of other screen elements or in newly created screen content (such as a line that appears on the screen as the control object moves), obviating the need for a special symbol. In the disengaged mode, a cursor symbol is typically used to visualize the current cursor location. Alternatively or additionally, a screen element or portion presently co-located with the cursor (and thus the selected screen location) can change brightness, color, or some other property to indicate that it is being pointed at. However, in certain implementations, the symbol or other visual indication of the cursor location can be omitted so that the user has to rely on his own observation of the control object relative to the screen to estimate the screen location pointed at. (For example, in a shooter game, the player can have the option to shoot with or without a “virtual sight” indicating a pointed-to screen location.)

Discrimination between the engaged and disengaged modes can be achieved by tracking the control object relative to a virtual control construct such as a virtual plane (or, more generally, a virtual surface construct). In an implementation and by way of example, as illustrated in FIGS. 1A and 1B, a virtual control construct implemented by a virtual plane **100** can be defined in front of and substantially parallel to the screen **102**. When the control object **104** “touches” or “pierces” the virtual plane (i.e., when its spatial location coincides with, intersects, or moves beyond the virtual plane’s computationally defined spatial location), the cursor **106** and/or machine interface operates in the engaged mode (FIG. 1B); otherwise, the cursor and/or machine interface operates in the disengaged mode (FIG. 1A). To implement two or more distinct engaged modes, multiple virtual planes can be defined. For instance, a

13

drawing application can define two substantially parallel virtual planes at different distances from the screen. When the user, moving his finger towards the screen, pierces the first virtual plane, the user can be able to operate menus and controls within the application; when his finger pierces the second virtual plane, the finger's further (e.g., lateral) motions can be converted to line drawings on the screen. Two parallel virtual planes can also be used to, effectively, define a virtual control construct with a certain associated thickness (i.e., a "virtual slab"). Control object movements within that virtual slab can operate the cursor in the engaged mode, while movements on either side of the virtual slab correspond to the disengaged mode. A planar virtual control construct with a non-zero thickness can serve to avoid unintended engagement and disengagement resulting from inevitable small motions in and out of the virtual plane (e.g., due to the inherent instability of the user's hand and/or the user's perception of depth). The thickness can vary depending on one or more sensed parameters (e.g., the overall speed of the control object's motion; the faster the movements, the thicker the slice can be chosen to be).

Transitions between the different operational modes can, but need not, be visually indicated by a change in the shape, color (as in FIGS. 1A and 1B), or other visual property of the cursor or other displayable object and/or audio feedback. In some implementations, the cursor symbol indicates not only the operational mode, but also the control object's distance from the virtual control construct. For instance, the cursor symbol can take the form of a circle, centered at the cursor location, whose radius is proportional to (or otherwise monotonically increasing with) the distance between control object and virtual control construct, and which, optionally, changes color when switching from the disengaged mode into the engaged mode.

Of course, the system under control need not be a desktop computer. FIG. 1C-1 illustrates an implementation in which free-space gestures are used to operate a handheld tablet 110. The tablet 110 can be connected, e.g., via a USB cable 112 (or any other wired or wireless connection), to a motion-capture device 114 (such as for example, a dual-camera motion controller as provided by Leap Motion, Inc., San Francisco, CA or other interfacing mechanisms and/or combinations thereof) that is positioned and oriented so as to monitor a region where hand motions normally take place. For example, the motion-capture device 114 can be placed onto a desk or other working surface, and the tablet 110 can be held at an angle to that working surface to facilitate easy viewing of the displayed content. The tablet 110 can be propped up on a tablet stand or against a wall or other suitable vertical surface to free up the second hand, facilitating two-hand gestures. FIG. 1C-2 illustrates a modified tablet implementation, in which the motion-capture device 114 is integrated into the frame of the tablet 110.

The virtual surface construct need not be planar, but can be curved in space, e.g., to conform to the user's range of movements. FIG. 1D illustrates, for example, a cylindrical virtual surface construct 120 in front of an arrangement of three monitors 122, 124, 126, which can all be connected to the same computer. The user's finger motions can control screen content on any one of the screens, depending on the direction in which the finger 128 points and/or the portion of the virtual surface construct 120 that it pierces. Of course, other types of curved virtual surfaces constructs of regular (e.g., spherical) or irregular shape, or virtual surface constructs composed of multiple (planar or curved) segments, can also be used in combination with one or more screens. Further, in some implementations, the virtual control con-

14

struct is a virtual solid construct or a virtual closed surface (such as, e.g., a sphere, box, oriented ellipsoid, etc.) or portion thereof, having an interior (or, alternatively, exterior) that defines a three-dimensional engagement target. For instance, in an application that allows the user to manipulate a globe depicted on the screen, the virtual control construct can be a virtual sphere located at some distance in front of the screen. The user can be able to rotate the on-screen globe by moving his fingertips while they are touching or piercing the spherical virtual surface construct (from outside). To allow the user to manipulate the globe from inside, the spherical virtual surface construct can be defined as surrounding the user (or at least his hand), with its exterior serving as the engagement target. Engagement and disengagement of the control object need not necessarily be defined relative to a two-dimensional surface. Rather, in some implementations, the virtual control construct can be a virtual point construct along a virtual line (or line segment) extending from the control object, or a line within a plane extending from the control object.

The location and/or orientation of the virtual surface construct (or other virtual control construct) can be defined relative to the room and/or stationary objects (e.g., a screen) therein, relative to the user, relative to the device 114 or relative to some combination. For example, a planar virtual surface construct can be oriented parallel to the screen, perpendicular to the direction of the control object, or at some angle in between. The location of the virtual surface construct can, in some implementations, be set by the user, e.g., by means of a particular gesture recognized by the motion-capture system. To give just one example, the user can, with her index finger stretched out, have her thumb and middle finger touch so as to pin the virtual surface construct at a certain location relative to the current position of the index-finger-tip. Once set in this manner, the virtual surface construct can be stationary until reset by the user via performance of the same gesture in a different location.

In some implementations, the virtual surface construct is tied to and moves along with the control object, i.e., the position and/or orientation of the virtual surface construct are updated based on the tracked control object motion. This affords the user maximum freedom of motion by allowing the user to control the user interface from anywhere (or almost anywhere) within the space monitored by the motion-capture system. To enable the relative motion between the control object and virtual surface construct that is necessary for piercing the surface, the virtual surface construct follows the control object's movements with some delay. Thus, starting from a steady-state distance between the virtual surface construct and the control object tip in the disengaged mode, the distance generally decreases as the control object accelerates towards the virtual surface construct, and increases as the control object accelerates away from the virtual surface construct. If the control object's forward acceleration (i.e., towards the virtual surface construct) is sufficiently fast and/or prolonged, the control object eventually pierces the virtual surface construct. Once pierced, the virtual surface construct again follows the control object's movements. However, whereas, in the disengaged mode, the virtual surface construct is "pushed" ahead of the control object (i.e., is located in front of the control object tip), it is "pulled" behind the control object in the engaged mode (i.e., is located behind the control object tip). To disengage, the control object generally needs to be pulled back through the virtual surface construct with sufficient acceleration to exceed the surface's responsive movement.

In an implementation, an engagement target can be defined as merely the point where the user touches or pierces a virtual control construct. For example, a virtual point construct can be defined along a line extending from or through the control object tip, or any other point or points on the control object, located a certain distance from the control object tip in the steady state, and moving along the line to follow the control object. The line can, e.g., be oriented in the direction of the control object's motion, perpendicularly project the control object tip onto the screen, extend in the direction of the control object's axis, or connect the control object tip to a fixed location, e.g., a point on the display screen. Irrespective of how the line and virtual point construct are defined, the control object can, when moving sufficiently fast and in a certain manner, "catch" the virtual point construct. Similarly, a virtual line construct (straight or curved) can be defined as a line within a surface intersecting the control object at its tip, e.g., as a line lying in the same plane as the control object and oriented perpendicular (or at some other non-zero angle) to the control object. Defining the virtual line construct within a surface tied to and intersecting the control object tip ensures that the control object can eventually intersect the virtual line construct.

In an implementation, engagement targets defined by one or more virtual point constructs or virtual line (i.e., linear or curvilinear) constructs can be mapped onto engagement targets defined as virtual surface constructs, in the sense that the different mathematical descriptions are functionally equivalent. For example, a virtual point construct can correspond to the point of a virtual surface construct that is pierced by the control object (and a virtual line construct can correspond to a line in the virtual surface construct going through the virtual point construct). If the virtual point construct is defined on a line projecting the control object tip onto the screen, control object motions perpendicular to that line move the virtual point construct in a plane parallel to the screen, and if the virtual point construct is defined along a line extending in the direction of the control object's axis, control object motions perpendicular to that line move the virtual point construct in a plane perpendicular to that axis; in either case, control object motions along the line move the control object tip towards or away from the virtual point construct and, thus, the respective plane. Thus, the user's experience interacting with a virtual point construct can be little (or no) different from interacting with a virtual surface construct. Hereinafter, the description will, for ease of illustration, focus on virtual surface constructs. A person of skill in the art will appreciate, however, that the approaches, methods, and systems described can be straightforwardly modified and applied to other virtual control constructs (e.g., virtual point constructs or virtual linear/curvilinear constructs).

The position and/or orientation of the virtual surface construct (or other virtual control construct) are typically updated continuously or quasi-continuously, i.e., as often as the motion-capture system determines the control object location and/or direction (which, in visual systems, corresponds to the frame rate of image acquisition and/or image processing). However, implementations in which the virtual surface construct is updated less frequently (e.g., only every other frame, to save computational resources) or more frequently (e.g., based on interpolations between the measured control object positions) can be provided for in implementations.

In some implementations, the virtual surface construct follows the control object with a fixed time lag, e.g., between 0.1 and 1.0 second. In other words, the location of the virtual

surface construct is updated, for each frame, based on where the control object tip was a certain amount of time (e.g., 0.2 second) in the past. This is illustrated in FIG. 2, which shows the control object and the virtual surface construct (represented as a plane) at locations within a consistent coordinate system across the figures for various points in time according to various implementations. As depicted, the plane can be computationally defined as substantially perpendicular to the orientation of the control object (meaning that its normal is angled relative to the control object orientation by less than a certain small amount, e.g., less than 5°, and preferably smaller than 1°). Of course, the virtual plane need not necessarily be perpendicular to the orientation of the control object. In some implementations, it is, instead, substantially parallel to the screen, but still dynamically positioned relative to the control object (e.g., so as to remain at a certain distance from the control object tip, where distance can be measured, e.g., in a direction perpendicular to the screen or, alternatively, in the direction of the control object).

At a first point $t=t_0$ in time, when the control object is at rest, the virtual plane is located at its steady-state distance d in front of the control object tip; this distance can be, e.g., a few millimeters. At a second point $t=t_1$ in time—after the control object has started moving towards the virtual plane, but before the lag period has passed—the virtual plane is still in the same location, but its distance from the control object tip has decreased due to the control object's movement. One lag period later, at $t=t_1+\Delta t_{lag}$, the virtual plane is positioned the steady-state distance away from the location of the control object tip at the second point in time, but due to the control object's continued forward motion, the distance between the control object tip and the virtual plane has further decreased. Finally, at a fourth point in time $t=t_2$, the control object has pierced the virtual plane. One lag time after the control object has come to a halt, at $t=t_2+\Delta t_{lag}$, the virtual plane is again a steady-state distance away from the control object tip but now on the other side. When the control object is subsequently pulled backwards, the distance between its tip and the virtual plane decreases again ($t=t_3$ and $t=t_4$), until the control object tip emerges at the first side of the virtual plane ($t=t_5$). The control object can stop at a different position than where it started, and the virtual plane will eventually follow it and be, once more, a steady-state distance away from the control object tip ($t=t_6$). Even if the control object continues moving, if it does so at a constant speed, the virtual plane will, after an initial lag period to "catch up," follow the control object at a constant distance.

The steady-state distances in the disengaged mode and the engaged mode can, but need not be the same. In some implementations, for instance, the steady-state distance in the engaged mode is larger, such that disengaging from the virtual plane (i.e., "unlicking") appears harder to the user than engaging (i.e., "clicking") because it requires a larger motion. Alternatively or additionally, to achieve a similar result, the lag times can differ between the engaged and disengaged modes. Further, in some implementations, the steady-state distance is not fixed, but adjustable based on the control object's speed of motion, generally being greater for higher control object speeds. As a result, when the control object moves very fast, motions toward the plane are "buffered" by the rather long distance that the control object has to traverse relative to the virtual plane before an engagement event is recognized (and, similarly, backwards motions for disengagement are buffered by a long disengagement steady-state distance). A similar effect can also be achieved by decreasing the lag time, i.e., increasing the responsive-

ness of touch-surface position updates, as the control object speed increases. Such speed-based adjustments can serve to avoid undesired switching between the modes that can otherwise be incidental to fast control object movements.

In various implementations, the position of the virtual plane (or other virtual surface construct) is updated not based on a time lag, but based on its current distance from the control object tip. That is, for any image frame, the distance between the current control object tip position and the virtual plane is computed (e.g., with the virtual-plane position being taken from the previous frame), and, based thereon, a displacement or shift to be applied to the virtual plane is determined. In some implementations, the update rate as a function of distance can be defined in terms of a virtual “potential-energy surface” or “potential-energy curve.” In FIG. 3A, an exemplary such potential-energy curve **300** is plotted as a function of the distance of the virtual plane from the control object tip according to various implementations. The negative derivative **302** (or slope) of this curve, which specifies the update rate, i.e., the shift in the virtual plane’s position per frame (in arbitrary units), is shown in FIG. 3B. The minima of the potential-energy curve **300** determine the steady-state distances **304**, **306** to both sides of the control object; at these distances, the virtual plane is not updated at all. At larger distances, the virtual plane is attracted towards the control object tip, at a rate that generally increases with distance. For example, at point **308**, where the virtual plane is a positive distance d_1 away from the control object, a negative displacement or shift Δs_1 is applied to bring the virtual plane closer. Conversely, at point **310**, where the virtual plane has a negative distance d_2 from the control object tip (corresponding to piercing of the virtual plane, i.e., the engaged mode), a positive shift Δs_2 is applied to move the virtual plane closer to the control object. At distances below the steady-state distance (e.g., at point **312**), the virtual plane is repelled by the control object and driven back towards the steady state. The magnitude of the local maximum **314** between the two steady states determines the level of force or acceleration needed to cross from the disengaged to the engaged mode or back. In certain implementations, the potential-energy curve **300** is given an even more physical interpretation, and its negative slope is associated with an acceleration, i.e., a change in the velocity of the virtual plane, rather than a change in its position. In this case, the virtual plane does not immediately stop as it reaches a steady state, but oscillates around the steady state. To slow down the virtual plane’s motion and thereby stabilize its position, a friction term can be introduced into the physical model.

The potential-energy curve need not be symmetric, or course. FIG. 3C, for example, shows an asymmetric curve in which the steady-state distance in the engaged mode is larger than that in the disengaged mode, rendering disengagement harder. Further, as illustrated in FIG. 3D, the curve can have more than two (e.g., four) steady states **320**, which can correspond to one disengaged and three engaged modes. The requisite force to transition between modes depends, again, on the heights of the local maxima **322** between the steady states. In some implementations, the curve abruptly jumps at the steady-state points and assumes a constant, higher value therebetween. In this case, which is illustrated in FIG. 3E, the position of the virtual plane is not updated whenever the control object tip is within the steady-state distance from the virtual plane on either side, allowing fast transitions between the modes. Accordingly, the potential-energy curve can take many other forms, which can be tailored to a desired engagement-disengagement force profile experienced by the

user. Moreover, the virtual plane can be updated in accordance with a two-dimensional potential-energy surface that defines the update rate depending on, e.g., the distances between the virtual plane and control object tip along various directions (as opposed to only one, e.g., the perpendicular and shortest, distance of the control object tip from the virtual plane). For example, the virtual plane can follow the control object differently for different relative orientations between the control object and the virtual plane, and each such relative orientation can correspond to a cross-section through the potential-energy surface. Two-dimensional potential-energy surfaces can also be useful to control position updates applied to a curved virtual surface construct.

Furthermore, the potential piercing energy need not, or not only, be a function of the distance from the control object tip to the virtual surface construct, but can depend on other factors. For example, in some implementations, a stylus with a pressure-sensitive grip is used as the control object. In this case, the pressure with which the user squeezes the stylus can be mapped to the piercing energy.

Whichever way the virtual surface construct is updated, jitter in the control object’s motions can result in unintentional transitions between the engaged and disengaged modes. While such modal instability can be combatted by increasing the steady-state distance (i.e., the “buffer zone” between control object and virtual surface construct), this comes at the cost of requiring the user, when she intends to switch modes, to perform larger movements that can feel unnatural. The trade-off between modal stability and user convenience can be improved by filtering the tracked control object movements. Specifically, jitter can be filtered out, based on the generally more frequent changes in direction associated with it, with some form of time averaging. Accordingly, in one implementation, a moving-average filter spanning, e.g., a few frames, is applied to the tracked movements, such that only a net movement within each time window is used as input for cursor control. Since jitter generally increases with faster movements, the time-averaging window can be chosen to likewise increase as a function of control object velocity (such as a function of overall control object speed or of a velocity component, e.g., perpendicular to the virtual plane). In another implementation, the control object’s previous and newly measured position are averaged with weighting factors that depend, e.g., on velocity, frame rate, and/or other factors. For example, the old and new positions can be weighted with multipliers of x and $(1-x)$, respectively, where x varies between 0 and 1 and increases with velocity. In one extreme, for $x=1$, the cursor remains completely still, whereas for the other extreme, $x=0$, no filtering is performed at all.

FIG. 4A summarizes representative methods for control-object-controlled cursor operation that utilize a virtual surface construct moving with the control object in accordance with various implementations. In the method implementation illustrated by FIG. 4A, a control object is tracked (**400**), based on computer vision or otherwise, to determine its position and/or orientation in space (typically within a detection zone proximate to the computer screen). Optionally, the tracked control object motion is computationally filtered to reduce jitter (**402**). Based on the tracked control object in conjunction with a definition of the virtual surface construct relative thereto, the position and/or orientation of the virtual surface construct are then computed (**404**). In implementations where the virtual surface construct is updated based on a control object position in the past, it can initially take a few control object tracking cycles (e.g.,

frames in image-based tracking) before the first position of the virtual surface construct is established; thereafter, the virtual surface construct can be updated every cycle. In implementations where the virtual surface construct is shifted from cycle to cycle based on its instantaneous distance from the control object tip, the position of the virtual surface construct can be initiated arbitrarily, e.g., such that the virtual surface construct starts a steady-state distance away from the control object. Following computation of the virtual surface construct, the current operational mode (engaged or disengaged) is identified based on a determination whether the control object touches or pierces the virtual surface construct or not (406). Further, the current cursor position is calculated, typically from the control object's position and orientation relative to the screen (408). (This step can be performed prior to, or in parallel with, the computations of the virtual surface construct.) Based on the operational mode and cursor position, the screen content is then updated (410), e.g., to move the cursor symbol or re-arrange other screen content. Steps 400-410 are executed in a loop as long as the user interacts with the system via free-space control object motions.

In some implementations, temporary piercing of the virtual surface construct—i.e., a clicking motion including penetration of the virtual surface construct immediately followed by withdrawal from the virtual surface construct—switches between modes and locks in the new mode. For example, starting in the disengaged mode, a first click event can switch the control object into the engaged mode, where it can then remain until the virtual surface construct is clicked at again.

Further, in some implementations, the degree of piercing (i.e., the distance beyond the virtual surface construct that the control object initially reaches, before the virtual surface construct catches up) is interpreted as an intensity level that can be used to refine the control input. For example, the intensity (of engagement) in a swiping gesture for scrolling through screen content can determine the speed of scrolling. Further, in a gaming environment or other virtual world, different intensity levels when touching a virtual object (by penetrating the virtual surface construct while the cursor is positioned on the object as displayed on the screen) can correspond to merely touching the object versus pushing the object over. As another example, when hitting the keys of a virtual piano displayed on the screen, the intensity level can translate into the volume of the sound created. Thus, touching or engagement of a virtual surface construct (or other virtual control construct) can provide user input beyond the binary discrimination between engaged and disengaged modes.

FIGS. 4B and 4B-1 illustrate at a higher conceptual level various methods for controlling a machine-user interface using free-space gestures or motions performed by a control object. The method involves receiving information including motion information for a control object (420). Further, it includes determining from the motion information whether the motion corresponds to an engagement gesture (422). This determination can be made by determining whether an intersection occurred between the control object and a virtual control construct (424); whether a dis-intersection of the control object from the at least one virtual control construct occurred (426); and/or whether motion of the control object occurred relative to at least one virtual control construct (428). Further, the determination can involve determining, from the motion information, one or more engagement attributes (e.g., a potential energy) defining an engagement gesture (430), and/or identifying an engage-

ment gesture by correlating the motion information to one of a plurality of engagement gestures based in part upon one or more of motion of the control object, occurrence of any of an intersection, a dis-intersection or a non-intersection of the control object with the virtual control construct, and the set of engagement attributes (432). Once an engagement gesture has been recognized, the user-interface control to which the gesture applies (e.g., a control associated with an application or an operating environment, or a special control) is selected or otherwise determined (434). The control can then be manipulated according to the gesture (436).

As will be readily apparent to those of skill in the art, the methods described above can be readily extended to the control of a user interface with multiple simultaneously tracked control objects. For instance, both left and right index fingers of a user can be tracked, each relative to its own associated virtual touch surface, to operate to cursors simultaneously and independently. As another example, the user's hand can be tracked to determine the positions and orientations of all fingers; each finger can have its own associated virtual surface construct (or other virtual control construct) or, alternatively, all fingers can share the same virtual surface construct, which can follow the overall hand motions. A joint virtual plane can serve, e.g., as a virtual drawing canvas on which multiple lines can be drawn by the fingers at once.

In an implementation and by way of example, one or more control parameter(s) and the control object are applied to some control mechanism to determine the distance of the virtual control construct to a portion of the control object (e.g., tool tip(s), point(s) of interest on a user's hand or other points of interest). In some implementations, a lag (e.g., filter or filtering function) is introduced to delay, or modify, application of the control mechanism according to a variable or a fixed increment of time, for example. Accordingly, implementations can provide enhanced verisimilitude to the human-machine interaction, and/or increased fidelity of tracking control object(s) and/or control object portion(s).

In one example, the control object portion is a user's finger-tip. A control parameter is also the user's finger-tip. A control mechanism includes equating a plane-distance between virtual control construct and finger-tip to a distance between finger-tip and an arbitrary coordinate (e.g., center (or origin) of an interaction zone of the controller). Accordingly, the closer the finger-tip approaches to the arbitrary coordinate, the closer the virtual control construct approaches the finger-tip.

In another example, the control object is a hand, which includes a control object portion, e.g., a palm, determined by a "palm-point" or center of mass of the entire hand. A control parameter includes a velocity of the hand, as measured at the control object portion, i.e., the center of mass of the hand. A control mechanism includes filtering forward velocity over the last one (1) second. Accordingly, the faster the palm has recently been travelling forward, the closer the virtual control construct approaches to the control object (i.e., the hand).

In a further example, a control object includes a control object portion (e.g., a finger-tip). A control mechanism includes determining a distance between a thumb-tip (e.g., a first control object portion) and an index finger (e.g., a second control object portion). This distance can be used as a control parameter. Accordingly, the closer the thumb-tip and index-finger, the closer the virtual control construct is determined to be to the index finger. When the thumb-tip and index finger touch one another, the virtual control construct is determined to be partially pierced by the index finger. A

lag (e.g., filter or filtering function) can introduce a delay in the application of the control mechanism by some time-increment proportional to any quantity of interest, for example horizontal jitter (i.e., the random motion of the control object in a substantially horizontal dimension). Accordingly, the greater the shake in a user's hand, the more lag will be introduced into the control mechanism.

User-interface control via free-space motions relies generally on a suitable motion-capture device or system for tracking the positions, orientations, and motions of one or more control objects. For a description of tracking positions, orientations, and motions of control objects, reference can be had to U.S. patent application Ser. No. 13/414,485, filed on Mar. 7, 2012, the entire enclosure of which is incorporated herein by reference. In various implementations, motion capture can be accomplished visually, based on a temporal sequence of images of the control object (or a larger object of interest including the control object, such as the user's hand) captured by one or more cameras. In one implementation, images acquired from two (or more) vantage points are used to define tangent lines to the surface of the object and approximate the location and shape of the object based thereon, as explained in more detail below. Other vision-based approaches that can be used in implementations include, without limitation, stereo imaging, detection of patterned light projected onto the object, or the use of sensors and markers attached to or worn by the object (such as, e.g., markers integrated into a glove) and/or combinations thereof. Alternatively or additionally, the control object can be tracked acoustically or ultrasonically, or using inertial sensors such as accelerometers, gyroscopes, and/or magnetometers (e.g., MEMS sensors) attached to or embedded within the control object. Implementations can be built employing one or more of particular motion-tracking approaches that provide control object position and/or orientation (and/or derivatives thereof) tracking with sufficient accuracy, precision, and responsiveness for the particular application.

FIGS. 5A and 5B illustrate an exemplary system for capturing images and controlling a machine based on motions relative to a virtual control construct according to various implementations. As shown in FIG. 5A, the system includes motion-capture hardware including two video cameras 500, 502 that acquire a stream of images of a region of interest 504 from two different vantage points. The cameras 500, 502 are connected to a computer 506 that processes these images to infer three-dimensional information about the position and orientation of a control object 508, or a larger object of interest including the control object (e.g., a user's hand), in the region of interest 504, and computes suitable control signals to the user interface based thereon. The cameras can be, e.g., CCD or CMOS cameras, and can operate, e.g., in the visible, infrared (IR), or ultraviolet wavelength regime, either by virtue of the intrinsic sensitivity of their sensors primarily to these wavelengths, or due to appropriate filters 510 placed in front of the cameras. In some implementations, the motion-capture hardware includes, co-located with the cameras 500, 502, one or more light sources 512 that illuminate the region of interest 504 at wavelengths matching the wavelength regime of the cameras 500, 502. For example, the light sources 512 can be LEDs that emit IR light, and the cameras 500, 502 can capture IR light that is reflected off the control object and/or objects in the background. Due to the inverse-square dependence of the illumination intensity on the distance between the light sources 512 and the illuminated object, foreground objects such as the control object generally appear signifi-

cantly brighter in the images than background objects, aiding in intensity-based foreground/background discrimination. In some implementations, the cameras 500, 502 and light sources 512 are disposed below the control object to be tracked and point upward. For example, they can be placed on a desk to capture hand motions taking place in a spatial region above the desk, e.g., in front of the screen. This location can be optimal both for foreground/background discrimination (because the background is in this case typically the ceiling and, thus, far away) and for discerning the control object's direction and tip position (because the usual pointing direction will lie, more or less, in the image plane).

The computer 506 processing the images acquired by the cameras 500, 502 can be a suitably programmed general-purpose computer. As shown in FIG. 5B, it can include a processor (or CPU) 520, associated system memory 522 (typically volatile memory, e.g., RAM), one or more permanent storage devices 524 (such as hard disks, CDs, DVDs, memory keys, etc.), a display screen 526 (e.g., an LCD screen or CRT monitor), input devices (such as a keyboard and, optionally, a mouse) 528, and a system bus 530 that facilitates communication between these components and, optionally via a dedicated interface, with the cameras 500, 502 and/or other motion-capture hardware. The memory 522 can store computer-executable instructions, conceptually illustrated as a group of modules and programmed in any of various suitable programming languages (such as, e.g., C, C++, Java, Basic, Python, Pascal, Fortran, assembler languages, etc.), that control the operation of the CPU and provide the requisite computational functionality for implementing methods in accordance herewith. Specifically, in addition to an operating system 532 that stores low-level system functions (such as memory allocation and file management) and one or more end-user applications 534 (such as, e.g., web browsers, office applications, or video games), the memory can store modules for image processing and control object tracking, computation of the virtual control construct and determination of the operational mode, and cursor operation and user-interface control.

The image-processing and tracking module 536 can analyze pairs of image frames acquired by the two cameras 500, 502 (and stored, e.g., in image buffers in memory 522) to identify the control object (or an object including the control object or multiple control objects, such as a user's hand) therein (e.g., as a non-stationary foreground object) and detect its edges. Next, the module 536 can, for each pair of corresponding rows in the two images, find an approximate cross-section of the control object by defining tangent lines on the control object that extend from the vantage points (i.e., the cameras) to the respective edge points of the control object, and inscribe an ellipse (or other geometric shape defined by only a few parameters) therein. The cross-sections can then be computationally connected in a manner that is consistent with certain heuristics and known properties of the control object (e.g., the requirement of a smooth surface) and resolves any ambiguities in the fitted ellipse parameters. As a result, the control object is reconstructed or modeled in three dimensions. This method, and systems for its implementation, are described in more detail in U.S. patent application Ser. No. 13/414,485, filed on Mar. 7, 2012, the entire enclosure of which is incorporated herein by reference. A larger object including multiple control objects can similarly be reconstructed with respective tangent lines and fitted ellipses, typically exploiting information of internal constraints of the object (such as a maximum physical

separation between the fingertips of one hand). The image-processing and tracking module **534** can, further, extract relevant control object parameters, such as tip positions and orientations as well as velocities, from the three-dimensional model. In some implementations, this information can be inferred from the images at a lower level, prior to or without the need for fully reconstructing the control object. These operations are readily implemented by those skilled in the art without undue experimentation. In some implementations, a filter module **538** receives input from the image-processing and tracking module **564**, and smoothens or averages the tracked control object motions; the degree of smoothing or averaging can depend on a control object velocity as determined by the tracking module **536**.

An engagement-target module **540** can receive tracking data about the control object from the image-processing and tracking module **536** and/or the filter module **538**, and use that data to compute a representation of the virtual control construct, i.e., to define and/or update the position and orientation of the virtual control construct relative to the control object (and/or the screen); the representation can be stored in memory in any suitable mathematical form. A touch-detection module **542** in communication with the engagement-target module **540** can determine, for each frame, whether the control object touches or pierces the virtual control construct. A cursor module **544** can, based on tracking data from the image-processing and tracking module **536**, determine a cursor location on the screen (e.g., as the projection of the control object tip onto the screen). The cursor module **544** can also include a visualization component that depicts a cursor at the computed location, preferably in a way that discriminates, based on output from the touch-detection module **542**, between the engaged and disengaged mode (e.g., by using different colors). The visualization component of the cursor module **544** can also modify the cursor appearance based on the control object distance from the virtual control construct; for instance, the cursor can take the form of a circle having a radius proportional to the distance between the control object tip and the virtual control construct. A user-interface control module **546** can map detected motions in the engaged mode into control input for the applications **534** running on the computer **506**. Collectively, the end-user application **534**, user-interface control module **546**, and cursor module **544** can compute the screen content, i.e., an image for display on the screen **526**, which can be stored in a display buffer (e.g., in memory **522** or in the buffer of a GPU included in the system).

The functionality of the different modules can, of course, be grouped and organized in many different ways, as a person of skill in the art would readily understand. Further, it need not necessarily be implemented on a single computer, but can be distributed between multiple computers. For example, the image-processing and tracking functionality of module **536** can be provided by a separate computer in communication with the computer on which the end-user applications controlled via free-space control object motions are executed. In one exemplary implementation, the cameras **500**, **502**, light sources **512**, and computational facility for image-processing and tracking are integrated into a single motion-capture device (which, typically, utilizes an application-specific integrated circuit (ASIC) or other special-purpose computer for image-processing). In another exemplary implementation, the camera images are sent from a client terminal over a network to a remote server computer for processing, and the tracked control object positions and orientations are sent back to the client terminal as input into the user interface. Implementations can be realized using

any number and arrangement of computers (broadly understood to include any kind of general-purpose or special-purpose processing device, including, e.g., microcontrollers, ASICs, programmable gate arrays (PGAs), or digital signal processors (DSPs) and associated peripherals) executing the methods described herein, an any implementation of the various functional modules in hardware, software, or a combination thereof.

Computer programs incorporating various features or functionality described herein can be encoded on various computer readable storage media; suitable media include magnetic disk or tape, optical storage media such as compact disk (CD) or DVD (digital versatile disk), flash memory, and any other non-transitory medium capable of holding data in a computer-readable form. Computer-readable storage media encoded with the program code can be packaged with a compatible device or provided separately from other devices. In addition, program code can be encoded and transmitted via wired, optical, and/or wireless networks conforming to a variety of protocols, including the Internet, thereby allowing distribution, e.g., via Internet download and/or provided on-demand as web-services.

The systems and methods described herein can find application in a variety of computer-user-interface contexts, and can replace mouse operation or other traditional means of user input as well as provide new user-input modalities. Free-space control object motions and virtual-touch recognition can be used, for example, to provide input to commercial and industrial legacy applications (such as, e.g., business applications, including Microsoft Outlook™; office software, including Microsoft Office™, Windows™, Excel™, etc.; graphic design programs; including Microsoft Visio™ etc.), operating systems such as Microsoft Windows™; web applications (e.g., browsers, such as Internet Explorer™); other applications (such as e.g., audio, video, graphics programs, etc.), to navigate virtual worlds (e.g., in video games) or computer representations of the real world (e.g., Google street View™), or to interact with three-dimensional virtual objects (e.g., Google Earth™). FIGS. 6A-9B illustrate various exemplary control inputs achievable with free-space hand motions and gestures when using systems and methods in accordance herewith.

An example of a compound gesture will be illustrated with reference to an implementation illustrated by FIGS. 6A-6D. These diagrams are merely an example; one of ordinary skill in the art would recognize many other variations, alternatives, and modifications. FIG. 6A illustrates a system **100a** comprising wired and/or wirelessly communicatively coupled components of a tower **602a**, a display device **604a**, a keyboard **606a** and optionally a tactile pointing device (e.g., mouse, or track ball) **608a**. In some implementations, computing machinery of tower **602a** can be integrated into display device **604a** in an "all in one" configuration. A position and motion sensing device (e.g., **600a-1**, **600a-2** and/or **600a-3**) comprises all or a portion of the non-tactile interface system of FIG. 1A, that provides for receiving non-tactile input based upon detected position(s), shape(s) and/or motion(s) made by a hand **104** and/or any other detectable object serving as a control object. The position and motion sensing device can be embodied as a stand-alone entity or integrated into another device, e.g., a computer, workstation, laptop, notebook, smartphone, tablet, smart watch or other type of wearable intelligent device(s) and/or combinations thereof. Position and motion sensing device can be communicatively coupled with, and/or integrated within, one or more of the other elements of the system, and can interoperate cooperatively with

25

component(s) of the system **100a**, to provide a non-tactile interface capabilities, such as illustrated by the non-tactile interface system **100** of FIG. 1A.

The motion sensing device (e.g., **600a-1**, **600a-2** and/or **600a-3**) is capable of detecting position as well as motion of hands and/or portions of hands and/or other detectable objects (e.g., a pen, a pencil, a stylus, a paintbrush, an eraser, a virtualized tool, and/or a combination thereof), within a region of space **110a** from which it is convenient for a user to interact with system **100a**. Region **110a** can be situated in front of, nearby, and/or surrounding system **100a**. In some implementations, the position and motion sensing device can be integrated directly into display device **604a** as integrated device **600a-2** and/or keyboard **106a** as integrated device **600a-3**. While FIG. 6A illustrates devices **600a-1**, **600a-2** and **600a-3**, it will be appreciated that these are alternative implementations shown in FIG. 6A for clarity sake. Keyboard **606a** and position and motion sensing device are representative types of “user input devices.” Other examples of user input devices (not shown in FIG. 6A) can be used in conjunction with computing environment **100a**, such as for example, a touch screen, light pen, mouse, track ball, touch pad, data glove and so forth. Accordingly, FIG. 6A is representative of but one type of system implementation. It will be readily apparent to one of ordinary skill in the art that many system types and configurations are suitable for use in conjunction with various implementations.

Tower **102a** and/or position and motion sensing device and/or other elements of system **100a** can implement functionality to provide virtual control surface **600a** within region **110a** with which engagement gestures are sensed and interpreted to facilitate user interactions with system **602a**. Accordingly, objects and/or motions occurring relative to virtual control surface **600a** within region **110a** can be afforded differing interpretations than like (and/or similar) objects and/or motions otherwise occurring.

As illustrated in FIG. 6A control object **104** (happens to be a pointing finger in this example) is moving toward an “Erase” button being displayed on display **604a** by a user desiring to select the “Erase” button. Now with reference to FIG. 6B, control object **104** has moved triggered an engagement gesture by means of “virtually contacting”, i.e., intersecting virtual control surface **600a**. At this point, unfortunately, the user has suffered misgivings about executing an “Erase.” Since the “Erase” button has been engaged, however, mere withdrawal of control object **104** (i.e., a “dis-intersection”) will not undo the erase operation selected. Accordingly, with reference to FIG. 6C, the user makes a wiping motion with a second control object (i.e., the user’s other hand in this example) indicating that the user would like to cancel an operation that is underway. Motion by a second control object illustrates a “compound gesture” that includes two or more gestures, sequentially or simultaneously. Compound gestures can be performed using a single control object, or two or more control objects (e.g., one hand, two hands, one stylus and one hand, etc.). In the illustrated case, the point/select and the wipe are two gestures made by two different control objects (two hands) occurring contemporaneously. Now with reference to FIG. 6D, when the second part of the compound gesture is recognized, the Erase button is no longer highlighted, indicating that the button is now “unselected”. The user is free to withdraw the first control object from engagement with the virtual control surface without triggering an “Erase” operation.

26

FIGS. 7A and 7B illustrate a zooming action performed by two fingers (thumb and index finger) according to various implementations. These diagrams are merely an example; one of ordinary skill in the art would recognize many other variations, alternatives, and modifications. As illustrated by FIG. 7A, an image **706** (happens to be a web page feed) is being displayed by display **704**, by a browser or other application. To zoom in, the user commences a motion including engaging a virtual control construct (not shown) interposed between the user and display **704** at an engagement target approximately over the right most column being displayed. In FIG. 7B, the finger tips **104a**, **104b** of the user are moved away from each other. This motion is recognized by device **700** from differences in images captured of the control object portion **104a**, **104b** and determined to be an engagement gesture including a spreading motion of the thumb and index finger-tip in front of the screen using the techniques described hereinabove. The result of interpreting the engagement gesture is passed to an application (and/or to the OS) owning the display **704**. The application owning display **704** responds by zooming-in the image of display **704**.

FIGS. 8A and 8B show how a swiping gesture by a finger in engaged mode can serve to scroll through screen content according to various implementations. These diagrams are merely an example; one of ordinary skill in the art would recognize many other variations, alternatives, and modifications. As illustrated by FIG. 8A, an image **806** (happens to be of dogs in this example) is being displayed by display **804**. When the user commences a motion relative to and engaged with a virtual control construct (not shown) interposed between the user and display **804** (e.g., at an engagement target approximately over the left-most dog), the user’s gesture can be interpreted as a control input for the application displaying the images. For example, in FIG. 8B, the user has swiped a finger-tip **104a** from left to right. This motion is recognized by device **800** from differences in images captured of the control object portion **104a** and determined to be an engagement gesture including a swiping motion from left to right that pierces the virtual control construct using the techniques described hereinabove. The result of interpreting the engagement gesture is passed to the image application, which responds by scrolling the image on the display **804**. On the other hand, the same gesture performed without engaging the virtual control construct can be passed to the operating system and, for example, used to switch the display **804** between multiple desktops or trigger some other higher-level function. This is just one example of how engagement gestures, i.e., gestures performed relative to a virtual control construct (whether in the engaged or the disengaged mode, or changing between the modes), can be used to provide different types of control input.

FIGS. 9A and 9B show how the motion of a control object in free space in conjunction with a virtual plane (or a slice of a certain thickness) can provide writing with a virtual pen onto a virtual paper defined in space according to various implementations. These diagrams are merely an example; one of ordinary skill in the art would recognize many other variations, alternatives, and modifications. As shown in FIG. 9A, a user moves a tool **104b** (happens to be a stylus) in free space in front of a writing area being displayed on the screen of display **904** so as to pierce a virtual control construct (not shown) (happens to be a plane) interposed between the user and display **904**. This motion is recognized by device **900** from differences in images captured of the control object portion **104b** and determined to be an engagement gesture including placing a virtual pen onto a virtual paper of space,

27

and is reflected by the contents of display 904. Continuing motion of the stylus 104b in space by the user after engaging the virtual control plane is interpreted as writing with the stylus 104b on the virtual paper of space and is reflected by the contents of display 904. As shown in FIG. 9B, when the user dis-engages with the virtual control construct, the virtual pen is lifted from the virtual paper, completing the letter “D” in script matching the handwriting of the user in free space. Accordingly, implementations can enable, e.g., signature capture, free-hand drawings, etc.

Certain implementations were described above. It is, however, expressly noted that the described implementations are not limiting, nor exhaustive, but rather the intention is that additions and modifications to what was expressly described herein can be provided for in implementations readily apparent to one of ordinary skill having access to the foregoing. Moreover, it is to be understood that the features of the various implementations described herein are not mutually exclusive and can exist in various combinations and permutations, even if such combinations or permutations were not made expressly herein. The implementations described herein have been presented for purposes of illustration and are not intended to be exhaustive or limiting. Many variations and modifications are possible in light of the foregoing teaching. The implementations described herein as well as implementations apparent in view of the foregoing description are limited only by the following claims.

What is claimed is:

1. A computer-implemented method of controlling a machine user interface, comprising:

receiving information including motion information for a control object;

determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to at least one virtual control construct defined within a field of view of an image capturing device; determining a control to which the engagement gesture is applicable; and manipulating the control according to at least the motion information.

2. The method of claim 1, wherein determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to a virtual control construct defined within a field of view of an image capturing device comprises:

determining whether an intersection occurred between control object and at least one virtual control construct.

3. The method of claim 2, further comprising: determining from the motion information whether the engagement includes continued motion after intersection.

4. The method of claim 1, wherein determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to a virtual control construct defined within a field of view of an image capturing device comprises:

determining whether a dis-intersection of the control object from the at least one virtual control construct occurred.

5. The method of claim 1, wherein determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an

28

engagement gesture applied to a virtual control construct defined within a field of view of an image capturing device comprises:

determining whether motion of the control object occurred relative to at least one virtual control construct.

6. The method of claim 1, wherein determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to a virtual control construct defined within a field of view of an image capturing device comprises:

determining from the motion information one or more engagement attributes defining an engagement gesture.

7. The method of claim 6, wherein determining from the motion information one or more engagement attributes defining an engagement gesture includes:

determining a potential energy from the motion information.

8. The method of claim 1, wherein determining from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to a virtual control construct defined within a field of view of an image capturing device comprises:

identifying an engagement gesture by correlating motion information to at least one engagement gesture based at least upon one or more of:

motion of the control object, occurrence of any of an intersection, a dis-intersection or a non-intersection of the control object with the virtual control construct, and a set of engagement attributes.

9. The method of claim 1, wherein determining a control to which the engagement gesture is applicable includes:

selecting at least one of a control associated with an application, a control associated with an operating environment, and a special control.

10. The method of claim 1, wherein manipulating a control according to at least the motion information includes:

controlling a user interface in a first mode, and otherwise controlling the user interface in a second mode different from the first mode.

11. The method of claim 1, further comprising:

updating at least a spatial position of the at least one virtual control construct based at least in part on a spatial position of the control object determined from the motion information; thereby enabling the spatial position of the at least one virtual control construct to follow tracked motions of the control object.

12. A system for controlling a machine user interface via free-space motions of a control object tracked with an image capturing device, the system comprising:

a processor; and

memory storing:

(i) motion information for the control object, and

(ii) processor-executable instructions for causing the processor to:

determine from the motion information whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to at least one virtual control construct defined within a field of view of the image capturing device, determine a control to which the engagement gesture is applicable, and manipulate the control according to at least the motion information.

29

13. A non-transitory machine-readable medium, storing one or more instructions which, when executed by one or more processors, cause the one or more processors to:

determine from motion information received for a control object whether a motion of the control object is an engagement gesture according to an occurrence of an engagement gesture applied to at least one virtual control construct defined within a field of view of an image capturing device;

determine a control to which the engagement gesture is applicable; and

manipulate the control according to at least the motion information.

14. The method of claim 1, wherein the at least one virtual control construct includes at least one of:

virtual surface constructs, virtual linear constructs, virtual curvilinear constructs, virtual point constructs, or virtual solid constructs.

15. The method of claim 14, wherein the virtual surface constructs comprise at least one of a plane, a curved open

30

surface, a closed surface, a bounded open surface, and a predefined multi-dimensional virtual surface defined in two or three dimensions.

16. The method of claim 14, wherein the virtual linear constructs comprise at least one of a one-dimensional virtual line, and a predefined line segment defined in one, two, or three dimensions.

17. The method of claim 14, wherein the virtual curvilinear constructs comprise at least one of a curve, and a predefined curve segment defined in one, two, or three dimensions.

18. The method of claim 14, wherein the virtual point constructs comprises a predefined zero-dimensional virtual point defined in one, two, or three dimensions.

19. The method of claim 14, wherein the virtual solid constructs comprise at least one of a sphere, a cylinder, a cube, and a predefined three-dimensional virtual solid defined in three dimensions.

* * * * *